

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Subject Name: SOFTWARE ENGINEERING

Subject code: CS T55

UNIT V

User Interface Design and Testing: Characteristics of a good User Interface – Types – Fundamentals of Component based GUI Development – A User Interface Design methodology – Coding – Software Documentation – Testing – Unit Testing – Black Box testing – White Box testing – Debugging – Program Analysis tools – Integration testing – Testing Object Oriented programs – System Testing – Issues.

2 Marks

1. Identify five desirable characteristics of a user interface. (or) List the characteristics of a good user interface (Nov 2017)

- Speed of learning.
- Speed of use
- Speed of recall
- Error prevention
- Attractiveness
- Consistency
- Feedback
- Support for multiple skill levels
- Error recovery (undo facility)
- User guidance and on-line help

2. Differentiate between user guidance and online help system.

- Online help system help users to know about the operation of the software any time while using the software
- The guidance messages prompt the user regarding the options he has regarding the next command, and the status of the last command, etc.

3. Differentiate between a mode-based interface and the modeless interface.

- A modeless interface has only a single mode and all the commands are available all the time during the operation of software
- In a mode-based interface, different sets of commands can be invoked depending on the mode in which the system is.

4. Compare Graphical User Interface (GUI) with text-based user interface?

Graphical User Interface	Text-based User Interface
A. Multiple windows with	

different information can be simultaneously be displayed on the screen	Multiple windows with different information cannot be simultaneously be displayed on the screen
B. Iconic information representation and symbolic information manipulation is possible	Iconic information representation and symbolic information manipulation is not possible
C. Supports command selection using an attractive and user friendly menu selection system.	It is not attractive one
D. Graphics terminal with graphics capability is Expensive	A cheap alphanumeric display terminal

5. What are the User interface Types?

- Command language based interfaces
- Menu-based interfaces
- Direct manipulation interfaces

6. Give any three characteristics of Command language based interface

- A command language-based interface is based on designing a command language which the user can use to issue the commands
- It allows fast interaction with the computer and simplifies the input of complex commands.
- A command language- based interface can be made concise requiring minimal typing by the user

7. What are disadvantages of Command language-based interface?

- Difficult to learn and require the user to memorize the set of primitive commands
- Most users make errors while formulating commands in the command language
- Command language-based interfaces are not suitable for casual and inexperienced users

8. What are the issues raised in designing a command language-based interface?

- Reduce the number of primitive commands that a user has to remember
- Minimize the total typing required while issuing commands

9. What are the types of menus?

- Scrolling menu - enable the user to view and select the menu items that cannot be accommodated on the screen
- Walking menu -structure a large collection of menu items.
- Hierarchical menu - menu items are organized in a hierarchy or tree structure

10. Define iconic interface

A Direct manipulation interfaces present the interface to the user in the form of visual models.

11. What do you mean by component-based GUI development style?

- A development style based on widgets (window objects) is called component-based (or widget-based) GUI development style.

- Use widgets as building blocks is because they help users learn an interface fast

12. Give the necessity of component-based GUI development.

Component-based GUI development recognizes that every user interface can easily be built from a handful of predefined components such as menus, dialog boxes, forms, etc.

13. What is window in terms of GUI?

- A window is a rectangular area on the screen. A window can be considered to be a virtual screen, in the sense that it provides an interface to the user for carrying out independent activities, e.g. one window can be used for editing a program and another for drawing pictures, etc
- A window can be divided into two parts: client part, and non-client part.

What is meant by window management system?

- A window management system is a resource manager. It keeps track of the screen area resource and allocates it to the different windows that seek to use the screen. It simplifies the task of a GUI designer.
- A WMS consists of two parts a window manager, and a window system

14. Identify the responsibilities of a window manager.

The window manager is responsible for managing and maintaining the non-client area of a window.

Window manager manages the real-estate policy, provides look and feel of each individual window

15. Identify at least eight primary types of window objects.

Label widget, Container widget, Pop-up menu, Pull-down menu, Dialog boxes, Push button, Radio buttons, and Combo boxes

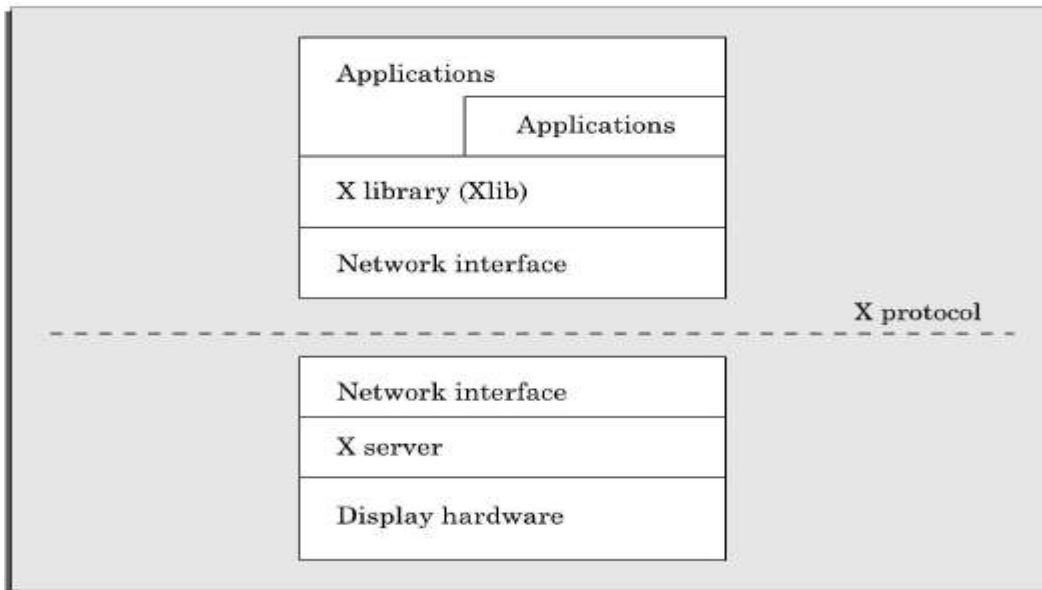
16. What is X Window?

- A windowing system, which is used to provide the basic framework for a GUI environment: drawing and moving windows on the display device and interacting with a mouse and keyboard.
- The X-window functions are low-level functions written in C language which can be called from application programs

17. Why is the X Window so popular?

- - The X-window system allows development of portable GUIs.
 - Applications developed using X-window system are device-independent.

18. Draw architecture of an X System.



19. Define X server.

- The X server runs on the hardware to which the display and keyboard are attached. The X server performs low-level graphics, manages windows, and user input functions.
- The X server controls access to a bit-mapped graphics display resource and manages it.

20. Define X protocol

- The X protocol defines the format of the requests between client applications and display servers over the network.
- The X protocol is designed to be independent of hardware, operating systems, underlying network protocol, and the programming language used.

21. What is X library(Xlib)?

An X Window System protocol, which is used to provide a set of about 300 utility routines for applications to call. These routines convert procedure calls into requests that are transmitted to the server.

22. Describe X toolkit(Xt)?

- A development software for building X Window applications.
- The X toolkit consists of two parts: the intrinsics and the widgets.
 - Widgets are predefined user interface components such as scroll bars, push buttons, etc. for designing GUIs.
 - Intrinsics are a set of about a dozen library routines that allow a programmer to combine a set of widgets into a user interface.

23. What is meant by visual programming?

A drag and drop style of program development, which has a number of visual objects (icons) representing the GUI components are provided by the programming environment.

Visual programming language greatly reduces the effort required to develop the interface.

Eg. Visual Basic, Visual C++, etc.

24. Differentiate between a user-centered design and a design by users.

- User-centered design is the **theme of almost all modern user interface design techniques**. However, user-centered design does not mean design by users. One should not get the users to design the interface, nor should one assume that the user's opinion of which design alternative is superior is always right.
- Users have good knowledge of the tasks they have to perform, they also **know whether they find an interface easy to learn** and use but they have less understanding and experience in GUI design than the GUI developers.

25. Explain implications of human cognition capabilities on user interface design.

- Limited memory
- Frequent task closure.
- Recognition rather than recall
- Procedural versus object-oriented

26. List out seven important steps needed to design a GUI methodology.

- Examine the use case model of the software. Interview, discuss, and review the GUI issues with the end-users.
- Task and object modeling
- Metaphor selection
- Interaction design and rough layout
- Detailed presentation and graphics design
- GUI construction
- Usability evaluation

27. Write check list for user interface inspection.

- Visibility of the system status
- Match between the system and the real world
- Undoing mistakes
- Consistency.
- Recognition rather than recall
- Support for multiple skill levels
- Aesthetic and minimalist design.
- Help and error messages
- Error prevention

28. Identify the necessity of coding standards.

- A coding standard gives a uniform appearance to the codes written by different engineers.
- It enhances code understanding.
- It encourages good programming practices.

29. Write about good coding standards?

- Rules for limiting the use of global
- Contents of the headers preceding codes for different modules
- Naming conventions for global variables, local variables, and constant identifiers
- Error return conventions and exception handling mechanisms
- Do not use a coding style that is too clever or too difficult to understand
- Avoid obscure side effects
- Do not use an identifier for multiple purposes
- The code should be well-documented
- The length of any function should not exceed 10 source lines
- Do not use goto statements

30. What is code review?

- A cost-effective strategy, which is used for reduction in coding errors and to produce high quality code.
- Types:
 - a. code inspection
 - b. code walk through.

32. Define Code Inspection

A technique, which is used to discover some common types of errors caused due to oversight and improper programming

33. Define Code walk through

An informal code analysis technique, which is used to eliminate successfully compiled and all syntax errors.

34. List out some classical programming errors during code inspection.

- Use of uninitialized variables.
- Jumps into loops.
- Non-terminating loops.
- Incompatible assignments.
- Array indices out of bounds.
- Improper storage allocation and de-allocation
- Mismatches between actual and formal parameter in procedure calls.
- Use of incorrect logical operators or incorrect precedence among operators.
- Improper modification of loop variables.
- Comparison of equality of floating point variables, etc

35. What is clean room testing?

A rigorous inspection process to test the code by other person (not programmer) to achieve zero-defect software.

36. What are the characteristic of clean room testing?

- Formal specification
- Incremental development
- Structured programming
- Statistical testing of the system
- Static verification

37. Explain the necessity of properly documenting software?

- Good documents enhance understandability and maintainability of a software product. They reduce the effort and time required for maintenance.
- Use documents help the users in effectively using the system.
- Good documents help in effectively handling the manpower turnover problem.
- Production of good documents helps the manager in effectively tracking the progress of the project

38. Differentiate between internal documentation and external documentation.

Internal documentation is the one in which various information regarding the program is enlisted in the program itself i.e. in the form of comments.

On the contrary, external documentation is the one that is prepared separately to inform the users about the system.

39. What is Program testing?

- Testing a program consists of providing the program with a set of test inputs (or test cases) and observing if the program behaves as expected.
- If the program fails to behave as expected, then the conditions under which failure occurs are noted for later debugging and correction

40. Explain the aim of testing.

The aim of the testing process is to identify all defects existing in a software product.

41. Differentiate between verification and validation.

Verification is the process of determining whether the output of one phase of software development conforms to that of its previous phase, whereas validation is the process of determining whether a fully developed system conforms to its requirements specification.

Verification focus on **phase containment of errors**, Validation aims to get **final product be error free**

42. Define failure.

A manifestation of an error (or defect or bug). But, the mere presence of an error may not necessarily lead to a failure

43. What is Test case?

This is the triplet [I,S,O], where I is the data input to the system, S is the state of the system at which the data is input, and O is the expected output of the system

44. Define Test suite?

The set of all test cases with which a given software product is to be tested

45. Why random selection of test cases is not effective.

All (or even most) of the errors in the system will be uncovered

46. Differentiate between functional testing (black-box testing) and structural testing(white-box testing).

Test cases are designed using only the functional specification of the software that is without any knowledge of the internal structure of the software called black-box testing (functional testing).

Designing test cases requires thorough knowledge about the internal structure of software called white-box testing (structural testing)

47. Differentiate between testing in the large and testing in the small.

- Software products are normally tested first at the individual component (or unit) level. This is referred to as testing in the small.
- After testing all the components individually, the components are slowly integrated and tested at each level of integration (integration testing).
- Finally, the fully integrated system is tested (called system testing). Integration and system testing are known as testing in the large.

48. What is unit testing?

Unit testing is undertaken after a module has been coded and successfully reviewed. Unit testing (or module testing) is the testing of different units (or modules) of a system in isolation

49. What is black box testing?

In the black-box testing, test cases are designed from an examination of the input/output values only and no knowledge of design or code is required

50. Identify equivalence classes for any given problem.

- If the input data values to a system can be specified by a range of values, then one valid and two invalid equivalence classes should be defined.
- If the input data assumes values from a set of discrete members of some domain, then one equivalence class for valid input values and another equivalence class for invalid input values should be defined

51. What is meant by boundary value analysis.

- **Boundary value analysis** is a software testing technique in which tests are designed to include representatives of **boundary values** in a range.
- Boundary value analysis-based test suite design involves designing test case using the values at the boundaries of different equivalence classes.

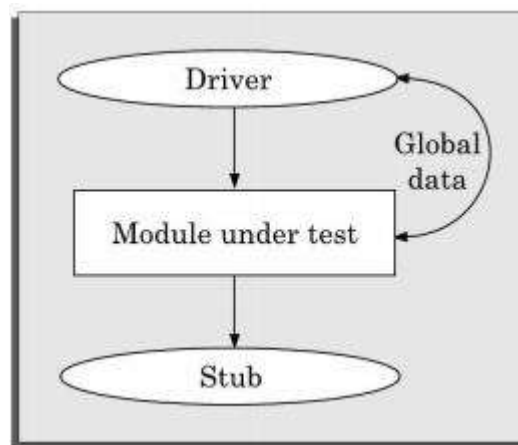
52. Design test cases corresponding to equivalence class testing and boundary value analysis for any given problem

Equivalence Class Partitioning:For a software that computes the square root of an input integer which can assume values in the range of 0 to 5000, there are three equivalence classes: The set of negative integers, the set of integers in the range of 0 and 5000, and the integers larger than 5000. Therefore, the test cases must include representatives for each of the three equivalence classes and a possible test set can be:
{-5,500,6000}.

Boundary Value Analysis: For a function that computes the square root of integer values in the range of 0 and 5000, the test cases must include the following values: {0, - 1,5000,5001}. Test cases for equivalence class testing and boundary value analysis for a problem. Let's consider a function that computes the square root of integer values in the range of 0 and 5000. For this particular problem, test cases corresponding to equivalence class testing and boundary value analysis have been found out earlier.

53. Differentiate stub and driver related to unit testing

- A stub procedure is a dummy procedure that has the same I/O parameters as the given procedure but has a highly simplified behavior.
For example, a stub procedure may produce the expected behavior using a simple table lookup mechanism.
- A driver module contains the nonlocal data structures accessed by the module under test, and would also have the code to call the different functions of the module with appropriate parameter values.



54. Define equivalence class partitioning?

The main idea behind defining equivalence classes of input data is that testing the code with any one value belonging to an equivalence class is as good as testing the code with any other value belonging to the same equivalence class.

55. Differentiate between stronger testing and complementary testing.

- One white-box testing strategy is said to be stronger than another strategy, if all types of errors detected by the first testing strategy is also detected by the second testing strategy, and the second testing strategy additionally detects some more types of errors.
- When two testing strategies detect errors that are different at least with respect to some types of errors, then they are called complementary.

56. Why do you need statement coverage strategy?

- The statement coverage strategy aims to design test cases so that every statement in a program is executed at least once.
- Unless a statement is executed, it is very hard to determine if an error exists in that statement

57. Why do you need branch coverage strategy?

- In the branch coverage-based testing strategy, test cases are designed to make each branch condition to assume true and false values in turn.
- Each edge of a program's control flow graph is traversed at least once called Edge testing, it is a stronger testing strategy compared to the statement coverage-based testing.

58. How to design condition coverage test cases for a code segment .

Test cases are designed to make each component of a composite conditional expression to assume both true and false values

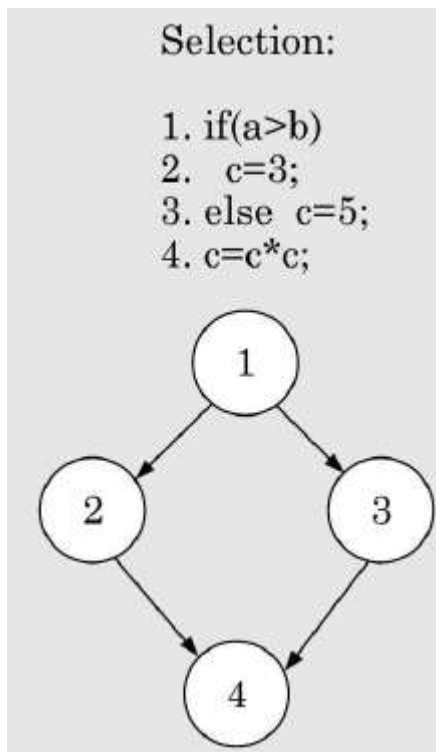
59. Design path coverage test cases for a code segment.

The path coverage-based testing strategy requires us to design test cases such that all linearly independent paths in the program are executed at least once.

60. Define Control Flow Graph(CFG)?

61. A control flow graph describes the sequence in which the different instructions of a program get executed.

62. Draw control flow graph for any program.



63. Identify the linear independent paths.

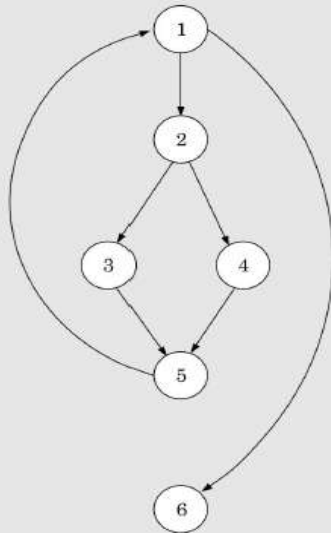
A set of paths for a given program is called linearly independent set of paths, if each path in the set introduces at least one new edge that is not included in any other path in the set.

If a set of paths is linearly independent of each other, then no path in the set can be obtained through any linear operations on the other paths in the set.

64. Compute cyclomatic complexity from any control flow graph.

```
int compute_gcd(int x, int y) {  
1 while(x!=y) {  
2   if(x>y) then  
3     x=x-y;  
4   else y=y-x;  
5 }  
6 return x;  
}
```

(a) An example program



(b) Control flow graph

- Given a control flow graph G of a program, the cyclomatic complexity $V(G)$ can be computed as:
 - $V(G) = E - N + 2$
 - where N is the number of nodes of the control flow graph and E is the number of edges in the control flow graph.
- For the CFG of example shown, $E=7$ and $N=6$. Therefore, the cyclomatic complexity = $7-6+2 = 3$.

65. Explain data flow-based testing.

- Data flow-based testing method selects test paths of a program according to the locations of the definitions and uses of different variables in a program.
- For a statement numbered S , let
$$\text{DEF}(S) = \{X/\text{statement } S \text{ contains a definition of } X\}, \text{ and}$$
$$\text{USES}(S) = \{X/\text{statement } S \text{ contains a use of } X\}$$

For the statement $S:a=b+c$;, $\text{DEF}(S) = \{a\}$. $\text{USES}(S) = \{b,c\}$.

- The definition of variable X at statement S is said to be live at statement $S1$, if there exists a path from statement S to statement $S1$ which does not contain any definition of X .

66. Explain mutation testing.

In mutation testing, the software is first tested by using an initial test suite built up from the different white box testing strategies.

After the initial testing is complete, mutation testing is taken up. The idea behind mutation testing is to make few arbitrary changes to a program at a time.

67. How to estimate of cyclomatic complexity of a program?

Cyclomatic complexity of a program is a measure of the psychological complexity or the level of difficulty in understanding the program

68. What are the steps followed in path coverage-based testing

1. Draw control flow graph
2. Determine $V(G)$
3. Determine the cyclomatic complexity. This gives the minimum number of test cases required to achieve path coverage.

4. Repeat

- a. Test using a randomly designed set of test cases.
- b. Perform dynamic analysis to check that path coverage achieved.
- c. Until at least 90 percent path coverage is achieved.

69. What is Debugging?

Debugging is the process of locating and fixing or bypassing bugs (errors) in computer program code or the engineering of a hardware device. To debug a program or hardware device is to start with a problem, isolate the source of the problem, and then fix it.

70. What are the three approaches of debugging.

- Brute Force Method
- Backtracking
- Cause Elimination Method
- Program Slicing

71. What is a Brute force method in debugging

- It is the least efficient method.
- The program is loaded with print statements to print the intermediate values with the hope that some of the printed values will help to identify the statement in error.
- It uses a symbolic debugger (also called a source code debugger)
- Values of different variables can be easily checked.
- Break points and watch points can be easily set to test the values of variables effortlessly.

72. Define Backtracking in debugging

In backtracking approach, beginning from the statement at which an error symptom has been observed, the source code is traced backwards until the error is discovered.

73. Explain three guidelines for effective debugging.

The following are some general guidelines for effective debugging:

- Debugging requires a thorough understanding of the program design.
- A common mistake done by the programmers are not attempt to fix the error but its symptoms. It may lead to redesign of the system.
- An error correction may not introduce new errors. Therefore after every round of error-fixing, regression testing must be carried out.

74. What is meant by a program analysis tool.

A program analysis tool means an automated tool that takes the source code or the executable code of a program as input and produces reports regarding several important characteristics of the program, such as its size, complexity, adequacy of commenting, adherence to programming standards, etc.

75. What are the categories of program analysis tool?

The program analysis tool is categorized into

- Static Analysis tools
- Dynamic Analysis tools

76. What is Static analysis tools (Kiviat Chart)?

Tools, which are used to summarize the results of analysis of every function in a polar chart. It shows the analyzed values for cyclomatic complexity, number of source lines, percentage of comment lines, etc..

77. What is Dynamic Analysis Tools?

Tools, which are used to evaluate several program characteristics based on analysis of the run-time

behavior of a program. It record and analyze the actual behavior of a program while it is being executed.

78. Give an important characteristic of a test suite computed by a dynamic analysis tool?

An important characteristic of a test suite computed by a dynamic analysis tool is the extent of coverage achieved by the test suite.

79. What is the objective of integration testing?

The objective of integration testing is to check whether the different modules of a program interface with each other properly.

80. What is Integration testing

In integration testing, individual software modules are combined and tested as a group.

It is to test the module interfaces, i.e. there are no errors in the parameter passing, when one module invokes another module.

Different modules of a system are integrated in a planned manner using an integration plan. The integration plan specifies the steps and the order in which modules are combined to realize the full system.

81. What are the approaches of Integration testing

The Integration test approaches are

- Big bang approach
- Top-down approach
- Bottom-up approach
- Mixed-approach

82. What is Big-bang integration testing?

A integration testing, in which all the modules makeup a system that are integrated in a single step.

83. What is Bottom-up integration testing?

Large software products are often made up of several subsystem. A Subsystems consist of many modules which communicate among each other through well-defined interface.

84. What is Top-down integration testing?

Top-down integration testing starts with the root module and one or two subordinate modules in the system.

85. What is Mixed integration testing?

The mixed(sandwiched) integration testing follows a combination of top-down and bottom-up testing approaches.

86. What is the difference between Phased and Incremental Integration Testing?

- In incremental integration testing, only one new module is added to the partially integrated system each time.
- In phased integration, a group of related modules are added to the partial system each time.

87. What are basic issues in testing object-oriented programs?

- a. What is a Suitable Unit for Testing Object-oriented Programs?
- b. Do various Object-orientation features make testing easy?
- c. Why are traditional techniques considered not satisfactory for testing object-oriented programs?
- d. Grey-Box testing of objects oriented programs?

88. What is Grey-Box Testing of Object-oriented programs?

For object-oriented programs, several types of test cases can be designed based on the design models of object-oriented programs. These are called the grey-box test cases

89. List out types of grey-box testing based on UML models?

- a. State-model-based testing
- b. Use case-based testing
- c. Class diagram-based testing
- d. Sequence diagram-based testing

90. What is State-model-based testing?

- a. State coverage : Each method of an object are tested at each state of the object
- b. State transition coverage: It is tested whether all transitions depicted in the state model work satisfactorily
- c. State transition path coverage: All transition paths in the state model are tested.

91. Describe Use case-based testing?

- a. Scenario coverage: Each use case consists of a mainline scenario and several alternate scenarios.
- b. For each use case, the mainline and all alternate sequences are tested to check if any errors show up

92. Describe Class diagram-based testing?

- a. Testing derived classes: All derived classes of the base class have to be instantiated and tested.
- b. Association testing: All association relations are tested
- c. Aggregation testing: Various aggregate objects are created and tested.

93. What is Sequence diagram-based testing?

- a. Method coverage: All methods depicted in the sequence diagrams are covered.
- b. Message path coverage: All message paths that can be constructed from the sequence diagrams are covered.

94. Give Integration Testing of Object-oriented Programs approaches.

Two main approaches

- i. Thread-based
- ii. Use based

95. What do you mean by Thread-based approach?

In this approach, all classes that need to collaborate to realize the behavior of a single use case are integrated and tested

96. Describe Use-based approach.

- a. Used-based integration begins by testing classes that either need no service from other classes or need services from at most a few other classes.
- b. After these classes have been integrated and tested, classes that use the services from the already integrated classes are integrated and tested
- c. This is continued till all the classes have been integrated and tested..

97. What is system testing?

System tests are designed to validate a fully developed system to assure that it meets its requirements. The test cases are therefore designed solely based on the SRS document.

98. What are the types of system testing?

The three main kinds of system testing are

- **Alpha Testing.** Alpha testing refers to the system testing carried out by the test team within the developing organization.
- **Beta testing.** Beta testing is the system testing performed by a select group of friendly customers.
- **Acceptance Testing.** Acceptance testing is the system testing performed by the customer to determine whether he should accept the delivery of the system.

99. Define Performance testing

- Performance testing is carried out to check whether the system needs the non-functional requirements identified in the SRS document.
- There are several types of performance testing.

- Stress testing
- Volume testing
- Configuration testing
- Compatibility testing
- Regression testing
- Recovery testing
- Maintenance testing
- Documentation testing
- Usability testing

100. What is Stress Testing(endurance testing)?

- Stress testing evaluates system performance when it is stressed for short periods of time.
- Stress tests are black box tests which are designed to impose a range of abnormal and even illegal input conditions so as to stress the capabilities of the software.
- Input data volume, input data rate, processing time, utilization of memory, etc. are tested beyond the designed capacity.
- For example, if the non-functional requirement specification states that the response time should not be more than 20 secs per transaction when 60 concurrent users are working, then during the stress testing the response time is checked with 60 users working simultaneously.

101. Define Volume Testing

- To check whether the data structures have been designed to successfully extraordinary situations.
- For example, a compiler might be tested to check whether the symbol table overflows when a very large program is compiled.

102. What is Configuration Testing?

- To analyze system behavior in various hardware and software configurations specified in the requirements.
- Sometimes systems are built in variable configurations for different users.
- For example, a minimal system to serve a single user, and other extension configurations to serve additional users.

103. Define Compatibility Testing

- It is required when the system interfaces with other types of systems.
- Aims to check whether the interface functions perform as required.
- For instance, if the system needs to communicate with a large database system to retrieve information, compatibility testing is required to test the speed and accuracy of data retrieval.

104. What do you mean by Regression Testing?

- It is required when the system being tested is an upgradation of an already existing system to fix some bugs or enhance functionality, performance, etc.
- Regression testing is the practice of running an old test suite after each change to the system or after each bug fix to ensure that no new bug has been introduced due to the change or the bug fix

105. What is Recovery Testing?

- Recovery testing tests the response of the system to the presence of faults, or loss of power, devices, services, data, etc.
- The system is subjected to the loss of the mentioned resources (as applicable and discussed in the SRS document) and it is checked if the system recovers satisfactorily.
- For example, the printer can be disconnected to check if the system hangs. Or, the power may be shut down to check the extent of data loss and corruption.

106. What is Maintenance Testing

- Addresses the diagnostic programs, and other procedures that are required to be developed to help maintenance of the system.

- It is verified that the artifacts exist and they perform properly.

107. Define Documentation Testing

- It is checked that the required user manual, maintenance manuals, and technical manuals exist and are consistent.
- If the requirements specify the types of audience for which a specific manual should be designed, then the manual is checked for compliance.

108. What is Usability Testing?

- Usability testing concerns checking the user interface to see if it meets all user requirements concerning the user interface.
- During usability testing, the display screens, report formats, and other aspects relating to the user interface requirements are tested.

109. Describe Error seeding

- Sometimes the customer might specify the maximum number of allowable errors that may be present in the delivered system.
- These are often expressed in terms of maximum number of allowable errors per line of source code.
- Error seed can be used to estimate the number of residual errors in a system.

110. What are General Issues associated with Testing?

- How to document the results of testing and how to perform regression testing
- Test summary Report:
 - A piece of documentation produced towards the end of testing .
 - It specifies what is the total number of tests that were applied to a subsystem, Out of the total number of tests how many tests were successful, and How many were unsuccessful, and the degree to which they were unsuccessful.
- Regressing testing is the practice of running an old test suite after each change to the system or after each bug fix to ensure that no new bug has been introduced due to the change or the bug fix.

111. Write short notes on white box testing. (Nov 2017)

White-box testing(Structural testing or glass box testing or clear box testing) :

- White box testing is done to test internals of the program. This is done by examining the program structure and by deriving test cases from the program logic.
- White-box testing is an important type of unit testing
- A white-box testing strategy can either be coverage-based or fault-based

11 Marks

1. Explain User interface design methodology (Nov 2017)

User Interface Design

Introduction

- In any software product, user interface portion is responsible for all interactions with the user.
- Almost every software product has a user interface:
 - can you think of any software product that does not have any user interface?
- In the early days of computer, no software product had any user interface:
 - all computers were batch systems
 - no interactions with users were supported.
- We know that things are very different now:
 - almost every software product is highly interactive.
- Users interact with a software product through its user interface:
 - user-interface portion of any software is directly relevant to the users.
 - many users judge a software from its user interface.
- User interface design:
 - a practical and important problem.
- Aesthetics apart, a difficult to use interface:
 - leads to higher levels of user errors
 - leads to user dissatisfaction.
- Users become particularly irritated when a system behaves in unexpected ways,
 - issued commands do not carry out actions according to intuitive expectations of users.
- A significant portion of the total development effort:
 - spent in developing the user interface.
- For many interactive applications:
 - as much as 50% of the total development effort is spent on the user interface part.
- If the user interface is not developed systematically:
 - effort required to develop the interface would increase tremendously :
 - leading to customer dissatisfaction.
- It is important to carefully study
 - user interface design concepts.
- Let us examine what a good interface is:
 - Unless we know what a good user interface really is,
 - we cannot possibly design one.

Characteristics of Good User Interfaces

- Speed of learning:
 - A good user interface should be simple to learn.
 - Most users are put off by complex syntax and semantics of the command issue procedures.

- Speed of learning
- A good user interface should not require
 - users to memorize commands.
- An important factor affecting the speed of learning is consistency:
 - Once, a user learns about a command,
 - should be able to use similar commands
 - even in different circumstances for carrying out similar actions.
- Users can learn about an interface faster, if it is based on:
 - day-to-day real life examples (aka metaphors)
 - use concepts with which users are already familiar.
- For example, interface of a text editor:
 - can use concepts similar to writing on paper:
 - such as cutting lines and paragraphs and pasting it at other places,
 - users can immediately relate to it.
- Also, learning is facilitated by:
 - intuitive command names
 - symbolic command issue procedures.
- Speed of use
- Speed of use is determined by:
 - the time and effort necessary to initiate and execute different commands.
 - The time and user effort necessary to execute different commands should be minimal.
- Examples of bad interfaces:
 - users required to type in lengthy commands
 - command issue involves moving the mouse to widely different areas of the screen
 - can slow down the operating speed of users.
- Speed of recall
- Once users learn how to use an interface:

- their speed of recall about using the interface should be maximized.
- The speed of recall is improved if the interface is based on:
 - metaphors
 - symbolic command issue procedures
 - intuitive command names.
- Error rate
- A good user interface should
 - minimize the scope of committing errors.
- Error rate can be easily measured:
 - count the errors committed by different users.
- Error monitoring can be automated:
 - instrument user interface with monitoring code
 - record the frequency and types of errors committed by different users
 - later display statistics of various kinds of user errors.
- How to reduce error possibilities?
- Consistency of command names,
- Consistency of command issue procedures,
- Consistency in behavior of similar commands
- Simplicity of command issue procedure, etc.
- Attractiveness
- A good user interface should be attractive:
 - An attractive user interface catches user attention and fancy.
 - In this respect,
 - graphics-based user interfaces have a definite advantage over text-based interfaces.
- Consistency
- Consistency of commands is very desirable.
 - allow users to generalize the knowledge about one aspect of the interface to another.

- Consistency helps in:
 - speed of learning,
 - speed of recall,
 - also helps in reduction of error rate.
- Feedback
- A good user interface must provide constant feedback to user actions:
 - For example, if any user request takes more than few seconds to process,
 - the user must be informed that his/her request is still being processed.
 - If possible, the user should be periodically informed about the progress made in processing the command.
- In the absence of any response from the computer for a long time:
 - a novice user might even start recovery/shutdown procedures in panic.
- Support for multiple skill levels
- A good user interface:
 - should support different levels of sophistication in command issue procedures:
 - users with different experience levels prefer different types of interfaces.
- Experienced users are more concerned about speed of command issue:
 - whereas novice users pay prime importance to usability aspects.
- Novice users discouraged by:
 - cryptic command names
 - complex command issue procedures.
- Elaborate commands:
 - slow down command issue procedure
 - put off experienced users.
- As users become familiar with an interface:
 - look for faster command issue procedures such as "hot-keys", "macros", etc.
- Error Recovery (Undo facility)
- All categories of users commit errors.

- A good interface should allow users to undo mistakes..
- Users are inconvenienced:
 - if they can not recover from even simple errors.
- User Guidance and On-line Help
- Users might need guidance
 - or seek help from the system.
- User Guidance is provided through two broad category of methods:
 - On-line help system
 - Guidance and error messages produced
 - in response to user actions.
- On-line Help System
- Generic help messages are not very useful:
 - on-line help messages should be tailored to the context in which help is invoked.
- A good on-line help should:
 - provide messages in context-dependent way.
 - keep track of what a user is doing
 - help messages should be tailored to user's experience level.
 - should take advantage of graphics capabilities of the screen
 - not just be a copy of the user manual.
- Guidance Messages
- The guidance messages should be carefully designed:
 - prompt the user:
 - next actions he/she might take,
 - current status of the system,
 - progress made so far in processing the command
- A good guidance system should have:
 - different levels of sophistication for different categories of users.

- users should have an option to turn off detailed messages.
- Error Messages
- Error messages should be polite.
- Error messages should not be associated with noise:
 - might embarrass the user.
- The messages should suggest how a given error can be rectified.
- If appropriate,
 - the user should be given the option of invoking on-line help
 - to find out more about the error situation.

Mode-Based versus Modeless Interface

- A mode is a state or collection of states:
 - in each state (or mode) different types of commands are available.
- Modeless interface:
 - same set of commands are available at all times.
- Mode-based interface:
 - different sets of commands are available depending on the mode in which the system is, i.e.
 - based on the past sequence of the user commands.
- A mode-based interface can be represented using a state transition diagram:
 - each node (state) of the state transition diagram represents a mode.
 - Every state (node) of the diagram
 - annotated with command names meaningful in that state.

Graphical User Interface (GUI) versus Text-Based User Interface

- In a GUI:
 - several windows with different information can be simultaneously displayed on user's screen.
 - This is perhaps the biggest advantage of GUI
 - user can simultaneously interact with several related items at any time
 - can even run many unrelated applications

- Iconic information representation and symbolic information manipulation is possible in a GUI.
- Symbolic information manipulation:
 - such as pulling an icon representing a file into a trash can for deleting
 - intuitively very appealing
 - user can instantly remember it.
- A GUI can support command selection:
 - using an attractive and user-friendly menu selection system.
- In a GUI, a pointing device can be used:
 - a mouse or a light pen to issue commands.
 - The use of a pointing device
 - makes command issue procedure simpler.
- On the flip side, a GUI requires:
 - special terminals with graphics capabilities
 - requires special input devices such a mouse.
- In contrast, a text-based interface:
 - can run even on cheap alphanumeric terminals:
 - Graphics terminals are usually much more expensive than alphanumeric terminals.
- Increasing availability of:
 - terminals with graphics capability
 - bit-mapped high-resolution displays
 - significant amount of local processing power.

2.Explain different types of User interfaces

Types of User Interfaces

- User interfaces can be classified into three categories:
 - Command language-based interface
 - Menu-based interface

- Direct manipulation interface
- Each category of interface has its advantages and disadvantages:
 - Modern applications sport a combination of all the three types of interfaces.
- Choice of Interface
- Which parts of the interface should be implemented using what type of interface?
 - No simple guidelines available
 - to a large extent depends on the experience and discretion of the designer.
 - a study of characteristics of the different interfaces would give us some idea.

Command Language-Based Interface

- As the name itself suggests:
 - Incorporates some language to form commands.
- Users frame the required commands in the language:
 - type them in whenever required.
- Design of command language interface
- Simple command language interface:
 - determine all the commands needed to be supported
 - assign unique names to the different commands.
- A more sophisticated command language interface:
 - allow users to compose primitive commands to form more complex commands.
 - Consider `cat x.dat|grep 123`
 - Like a programming language.

The facility to compose commands:

- dramatically reduces the number of command names users would have to remember.
- Commands can be made concise
 - requiring minimal typing by the user.
- allow faster interaction with the computer
- simplify input of complex commands.

Advantages of Command Language Interfaces

- Easy to develop:
 - compiler writing techniques are well developed.
- Can be implemented even on cheap alphanumeric terminals.
- Much more efficient:
 - compared to other types of interfaces.

Disadvantages of Command Language Interfaces

- Difficult to learn:
 - Require the user to memorize primitive commands.
- Require the user to type in commands.
- Users make errors while:
 - formulating commands in the command language
 - typing them in.
- All interactions are through key-board:
 - cannot take advantage of effective interaction devices such as a mouse.
 - For casual and inexperienced users,
 - command language interfaces are not suitable.
- Issues in Designing a Command Language Interface
- Design of a command language interface:
 - involves several issues.
- The designer has to decide
 - what mnemonics are to be used for the commands.
 - mnemonics should be meaningful
 - yet be concise to minimize the amount of typing required.
- The designer has to decide:
 - whether users will be allowed to redefine command names to suit their own preferences.

- Letting a user define his own mnemonics for various commands is a useful feature,
- but increases complexity of user interface development.
- Designer has to decide:
 - whether it should be possible to compose primitive commands to create more complex commands.
 - syntax and semantics of command composition options has to be clearly and unambiguously decided.
- The ability to combine commands is a powerful facility for experienced users:
 - but quite unnecessary for inexperienced users.

Menu-Based Interface

- Advantages of a menu-based interface over a command language interface:
 - users are not required to remember exact command names.
 - typing effort is minimal:
 - menu selections using a pointing device.
 - This factor becomes very important for the occasional users who cannot type fast.
- For experienced users:
 - menu-based interfaces is slower than command language interfaces
 - experienced users can type fast
 - also get speed advantage by composing simple commands into complex commands.
- Composition of commands in a menu-based interface is not possible.
 - actions involving logical connectives (and, or, all, etc.)
 - awkward to specify in a menu-based system.
- If the number of choices is large,
 - it is difficult to design a menu-based interface.
 - Even moderate sized software needs hundreds or thousands of menu choices.
- A major problem with the menu-based interface:
 - structuring large number of menu choices into manageable forms.

Structuring Menu Interface

- Any one of the following options is adopted to structure menu items.
 - Walking menu
 - Scrolling menu
 - Hierarchical menu
- Scrolling Menu
 - Used when the menu options are highly related.
 - For example text height selection in a word processing software.
 - Scrolling of menu items
 - lets the user to view and select the menu items that can not be accommodated on one screen.

Walking Menu

- Walking menu is commonly used to structure large menu lists:
 - when a menu item is selected,
 - it causes further menu items to be displayed adjacent to it in a submenu.
- A walking menu can successfully structure commands only if:
 - there are tens rather than hundreds of choices
 - each adjacently displayed menu does take up some screen space
 - the total screen area is after all limited.

Hierarchical menu

- Menu items are organized in a hierarchy or tree structure.
 - Selecting a menu item causes the current menu display to be replaced by an appropriate submenu.
 - One can consider the menu and its various submenu to form a hierarchical tree-like structure.
- Walking menu are a form of hierarchical menu:
 - practicable when the tree is shallow.
- Hierarchical menu can be used to manage large number of choices,
 - but, users face navigational problems
 - lose track of where they are in the menu tree.

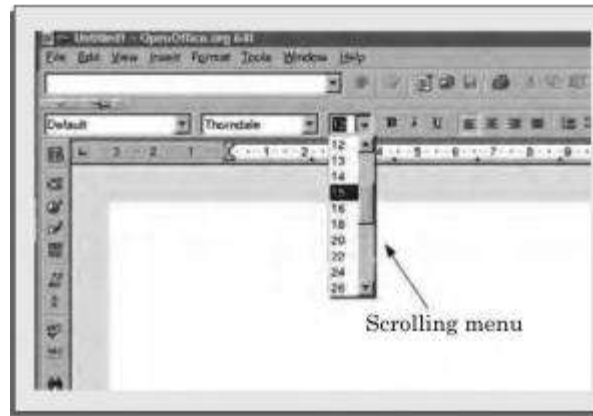


Fig.Font size selecting using scrolling menu

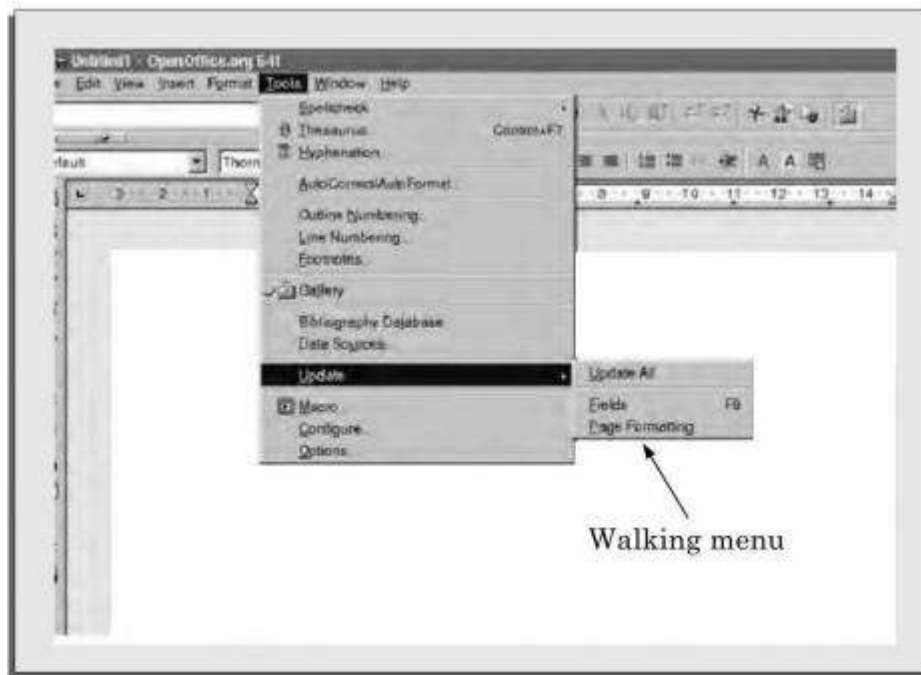


Fig. Example of walking menu

Direct Manipulation Interface

- Present information to the user as visual models or objects.
- Actions are performed on the visual representations of the objects,
e.g.pull an icon representing a file into an icon representing a trash box, for deleting the file.
- Direct manipulation interfaces are sometimes called as iconic interfaces.
- Important advantages of iconic interfaces:
 - icons can be recognized by users very easily,
 - icons are language-independent.

- However, experienced users consider direct manipulation interfaces too slow.
- It is difficult to form complex commands using a direct manipulation interface.
- For example, if one has to drag a file icon into a trash box icon for deleting a file:
 - to delete all files in a directory one has to perform this operation again and again
 - very easily done in a command language-interface by issuing a command `delete *.*`

3.Explain fundamentals of component-based GUI Development?

Fundamentals of Component-based GUI Development

Window

- A window is a rectangular area on the screen.
- A window can be considered to be a virtual screen, in the sense that it provides an interface to the user for carrying out independent activities, e.g. one window can be used for editing a program and another for drawing pictures, etc.
- A window can be divided into two parts: client part, and non-client part.
- The client area makes up the whole of the window, except for the borders and scroll bars.
- The client area is the area available to a client application for display.
- The non-client part of the window **determines the look and feel of the window**.
- The look and feel defines a basic behavior for all windows, such as creating, moving, resizing, and iconifying the windows. A basic window with its different parts is shown in fig.

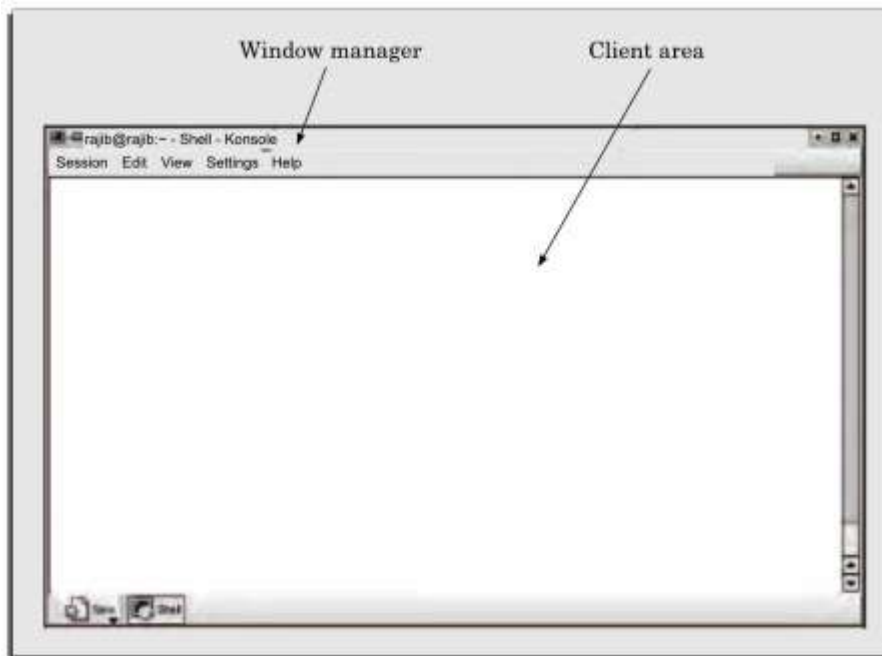


Fig: Window with client and user areas marked

Window Management System (WMS)

- A graphical user interface typically consists of a large number of windows. Therefore, it is necessary

to have some systematic way to manage these windows.

- Most graphical user interface development environments do this through a window management system (WMS).
- A window management system is primarily a resource manager. It keeps track of the screen area resource and allocates it to the different windows that seek to use the screen.
- From a broader perspective, a WMS can be considered as a user interface management system (UIMS) – which not only does resource management, but also provides the basic behavior to the windows and provides several utility routines to the application programmer for user interface development.
- A WMS simplifies the task of a GUI designer to a great extent by providing the basic behavior to the various windows such as move, resize, iconify, etc. as soon as they are created and by providing the basic routines to manipulate the windows from the application such as creating, destroying, changing different attributes of the windows, and drawing text, lines, etc.

A WMS consists of two parts

1. a window manager, and
2. a window system

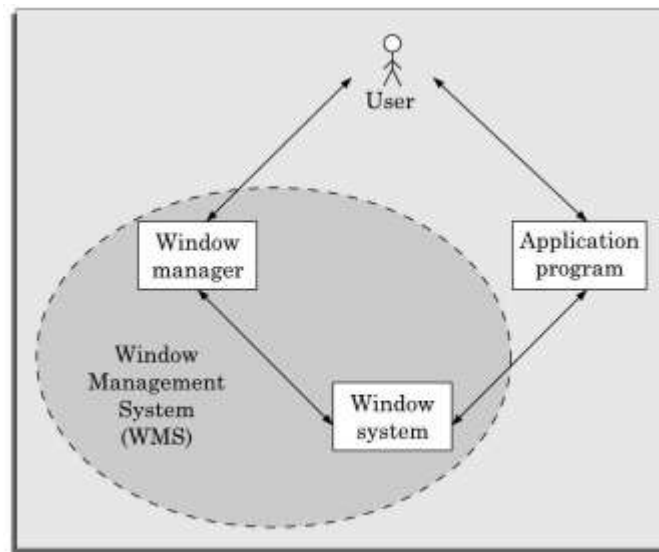


Fig.Window Management System

Window Manager and Window System

- Window manager is the component of WMS with which the end user interacts to do various window-related operations such as window repositioning, window resizing, iconification, etc.
- The window manager is built on the top of the window system in the sense that it makes use of various services provided by the window system.
- The window manager and not the window system determines how the windows look and behave.
- In fact, several kinds of window managers can be developed based on the same window system.
- The window manager can be considered as a special kind of client that makes use of the services (function calls) supported by the window system.
- The application programmer can also directly invoke the services of the window system to develop the user interface.
- The relationship between the window manager, window system, and the application program is shown in fig. This figure shows that the end-user can either interact with the application itself or with the window manager (resize, move, etc.) and both the application and the window manager invoke services

of the window manager.

Window Manager

- The window manager is responsible for managing and maintaining the non-client area of a window.
- Window manager manages the real-estate policy, provides look and feel of each individual window.

Types of widgets (window objects)

- Different interface programming packages support different widget sets. However, a surprising number of them contain similar kinds of widgets, so that one can think of a generic widget set which is applicable to most interfaces.
- The following widgets are representatives of this generic class.

Label widget. This is probably one of the simplest widgets. A labelwidget does nothing except to display a label, i.e. it does not have any other interaction capabilities and is not sensitive to mouse clicks. A label widget is often used as a part of other widgets.

Container widget. These widgets do not stand by themselves, but exist merely to contain other widgets. Other widgets are created as children of the container widget. When the container widget is moved or resized, its children widget also get moved or resized. A container widget has no callback routines associated with it.

Pop-up menu. These are transient and task specific. A pop-up menu appears upon pressing the mouse button, irrespective of the mouse position.

Pull-down menu. These are more permanent and general. You have to move the cursor to a specific location and pull down this type of menu.

Dialog boxes. We often need to select multiple elements from a selection list. A dialog box remains visible until explicitly dismissed by the user. A dialog box can include areas for entering text as well as values. If an apply command is supported in a dialog box, the newly entered values can be tried without dismissing the box. Through most dialog boxes ask you to enter some information, there are some dialog boxes which are merely informative, alerting you to a problem with your system or an error you have made. Generally, these boxes ask you to read the information presented and then click OK to dismiss the box.

Push button. A push button contains key words or pictures that describe the action that is triggered when you activate the button. Usually, the

action related to a push button occurs immediately when you click a push button unless it contains an ellipsis (...). A push button with an ellipsis generally indicates that another dialog box will appear.

Radio buttons. A set of radio buttons is used when only one option has to be selected out of many options. A radio button is a hollow circle followed by text describing the option it stands for. When a radio button is selected, it appears filled and the previously selected radio button from the group is unselected. Only one radio button from a group can be selected at any time. This operation is similar to that of the band selection buttons that were available in old radios.

Combo boxes. A combo box looks like a button until the user interacts with it. When the user presses or clicks it, the combo box displays a menu of items to choose from. Normally a combo box is used to display either one-of-many choices when space is limited, the number of choices is large, or when the

menu items are computed at run-time.

X-Window.

- The X-window functions are low-level functions written in C language which can be called from application programs. But only the very serious application designer would program directly using the X-windows library routines.
- Built on top of X-windows are higher-level functions collectively called Xtoolkit. Xtoolkit consists of a set of basic widgets and a set of routines to manipulate these widgets.
- One of the most widely used widget sets is X/Motif. Digital Equipment Corporation (DEC) used the basic X-window functions to develop its own look and feel for interface designs called DECWindows.

Popularity of X-Window

- One of the important reasons behind the extreme popularity of the X-window system is probably due to the fact that it allows development of portable GUIs.
 - Applications developed using X-window system are device-independent.
 - Also, applications developed using the X-window system become network independent in the sense that the interface would work just as well on a terminal connected anywhere on the same network as the computer running the application is.
 - Network-independent GUI operation has been schematically represented in the fig. Here, “A” is the computer application in which the application is running.
 - “B” can be any computer on the network from where interaction with the application can be made.
- Network-independent GUI was pioneered by the X-window system in the mid-eighties at MIT (Massachusetts Institute of Technology) with support from DEC (Digital Equipment Corporation).

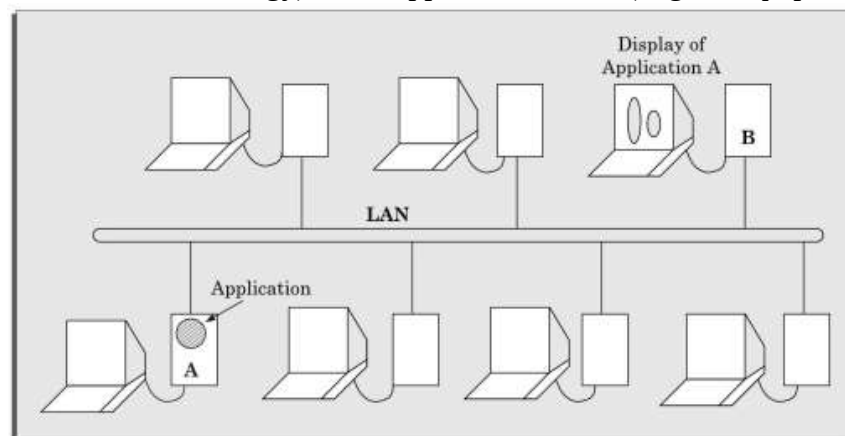
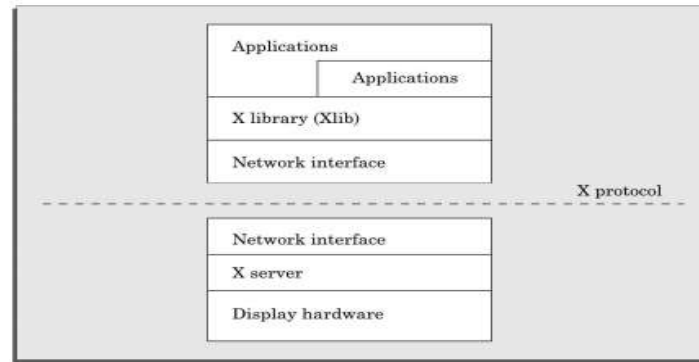


Fig. Network-independent GUI

- Now-a-days many user interface development systems support network-independent GUI development, e.g. the AWT and Swing components of Java.

Architecture of an X-System

- The X-architecture is pictorially depicted in fig.. The different terms used in this diagram are explained below



Architecture of the X-System

X-server. The X server runs on the hardware to which the display and keyboard are attached. The X server **performs low-level graphics, manages window, and user input functions.** The X server controls accesses to a bit-mapped graphics display resource and manages it.

X-protocol. The X protocol **defines the format of the requests between** client applications and display servers over the network. The X protocol is designed to be independent of hardware, operating systems, underlying network protocol, and the programming language used.

X-library (Xlib). The Xlib provides a set of about 300 utility routines for applications to call. These routines convert procedure calls into requests that are transmitted to the server. Xlib provides low level primitives for developing an user interface, such as displaying a window, drawing characteristics and graphics on the window, waiting for specific events, etc.

Xtoolkit (Xt). The Xtoolkit consists of two parts: the intrinsics and the widgets. We have already seen that widgets are predefined user interface components such as scroll bars, push buttons, etc. for designing GUIs. **Intrinsics** are a set of about a dozen library routines that allow **a programmer to combine a set of widgets into a user interface.** In order to develop a user interface, the designer has to put together the set of components (widgets) he needs, and then he needs to define the characteristics (called resources) and behavior of these widgets by using the intrinsic routines to complete the development of the interface. Therefore, developing an interface using Xtoolkit is much easier than developing the same interface using only X library.

Visual Programming

- Visual programming is **the drag and drop style of program development.** In this style of user interface development, a number of visual objects (icons) representing the GUI components are provided by the programming environment.
- The application programmer can easily develop the user interface **by dragging the required component types (e.g. menu, forms, etc.) from the displayed icons and placing them wherever required.**
- Thus, visual programming can be considered as program development through manipulation of several visual objects.
- Reuse of program components in the form of visual objects is an important aspect of this style of programming.
- Though popular for user interface development, this style of programming can be used for other applications such as Computer-Aided Design application (e.g. factory design), simulation, etc.
- User interface development using a visual programming language greatly reduces the effort required to develop the interface.

Examples of popular visual programming languages are Visual Basic, Visual C++, etc.

- Visual C++ provides tools for building programs with window-based user interfaces for Microsoft Windows environments.
- In Visual C++, menu bars, icons, and dialog boxes, etc. can be designed easily before adding them to program.
- These objects are called as resources. Shape, location, type, and size of the dialog boxes can be designed before writing any C++ code for the application.

Difference between user-centered design and design by users

- User-centered design is the theme of almost all modern user interface design techniques. However, user-centered design does not mean design by users. One should not get the users to design the interface, nor should one assume that the user's opinion of which design alternative is superior is always right.
- Users have good knowledge of the tasks they have to perform, they also know whether they find an interface easy to learn and use but they have less understanding and experience in GUI design than the GUI developers.

Implications of human cognition capabilities on user interface design

- An area of human-computer interaction where extensive research has been conducted is how human cognitive capabilities and limitations influence the way an interface should be designed. The following are some of the prominent issues extensively discussed in the literature.
- **Limited memory.** Humans can remember at most seven unrelated items of information for short periods of time. Therefore, the GUI designer should not require the user to remember too many items of information at a time. It is the GUI designer's responsibility to anticipate what information the user will need at what point of each task and to ensure that the relevant information is displayed for the user to see. Showing the user some information at some point, and then asking him to recollect that information in a different screen where they no longer see the information places a memory burden on the user and should be avoided wherever possible.
- **Frequent task closure.** Doing a task (except for very trivial tasks) requires doing several subtasks. When the system gives a clear feedback to the user that a task has been successfully completed, the user gets a sense of achievement and relief. The user can clear out information regarding the completed task from memory. This is known as task closure. When the overall task is fairly big and complex, it should be divided into subtasks, each of which has a clear subgoal which can be a closure point.
- **Recognition rather than recall.** Information recall incurs a larger memory burden on the users and is to be avoided as far as possible. On the other hand, recognition of information from the alternatives shown to him is more acceptable.
- **Procedural versus object-oriented.** Procedural designs focus on tasks, prompting the user in each step of the task, giving them few options for anything else. This approach is the best applied in situations where the tasks are narrow and well-defined or where the users are inexperienced, such as an ATM. An object-oriented interface on the other hand focuses on objects. This allows the users a wide range of options.

4. Explain GUI design methodology?

GUI design methodology

- GUI design methodology consists of the following important steps:
 - Examine the use case model of the software. Interview, discuss, and review the GUI issues with the end-users.
 - Task and object modeling
 - Metaphor selection
 - Interaction design and rough layout
 - Detailed presentation and graphics design
 - GUI construction
 - Usability evaluation
- The starting point for GUI design is the use case model. This captures the important tasks the users need to perform using the software. As far as possible, a user interface should be developed using one or more metaphors.
- Metaphors help in interface development at lower effort and reduced costs for training the users.
- Over time, people have developed efficient methods of dealing with some commonly occurring situations.
- These solutions are the themes of the metaphors. Metaphors can also be based on physical objects such as a visitor's book, a catalog, a pen, a brush, a scissor, etc.
- A solution based on metaphors is easily understood by the users, reducing learning time and training costs. Some commonly used metaphors are the following:
 - White board
 - Shopping cart
 - Desktop
 - Editor's work bench
 - White page
 - Yellow page
 - Office cabinet
 - Post box
 - Bulletin board
 - Visitor's book

Task and Object Modeling

- A task is a human activity intended to achieve some goals. Example of task goals can be:
 - reserve an airline seat
 - buy an item
 - transfer money from one account to another
 - book a cargo for transmission to an address

- A task model is an abstract model of the structure of a task. A task model should show the structure of the subtasks that the user needs to perform to achieve the overall task goal.
- Each task can be modeled as a hierarchy of subtasks.
- A task model can be drawn using a graphical notation similar to the activity network model. Tasks can be drawn as boxes with lines showing how a task is broken down into subtasks.
- An underlined task box would mean that no further decomposition of the task is required. An example decomposition of a task into subtasks is shown in fig.

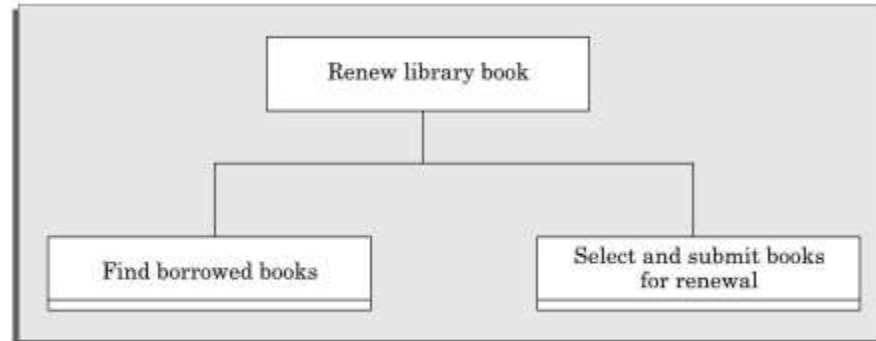
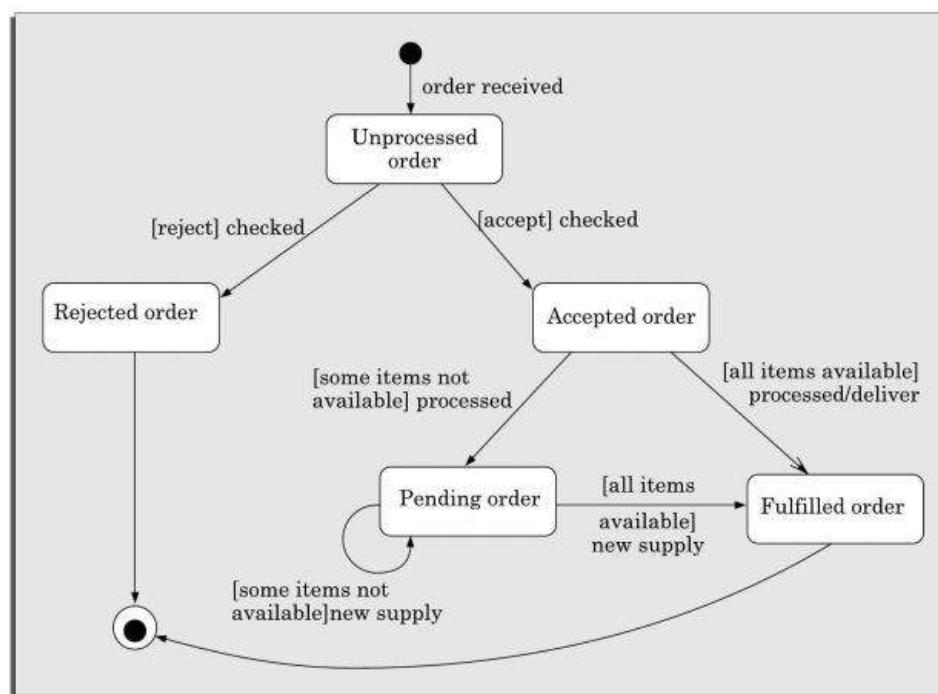


Fig.Decomposition of a task into subtasks



State chart diagram for an order object

Selecting a metaphor

- The first place one should look for while trying to identify the candidate metaphors is the set of parallels to objects, tasks, and terminologies of the use cases.
- If no obvious metaphors can be found, then the designer can fall back on the metaphors of the physical world of concrete objects.
- The appropriateness of each candidate metaphor should be tested by restating the objects and tasks of the user interface model in terms of the metaphor.
- Another criterion that can be used to judge metaphors is that the metaphor should be as simple as possible, the operations using the metaphor should be clear and coherent and it should fit with the users' 'common sense' knowledge.

- For example, it would indeed be very awkward and a nuisance for the users if the scissor metaphor is used to glue different items.
- **Example:** We need to develop the interface for the automation shop, where the users can examine the contents of the shop through a web interface and can order them.
- Several metaphors are possible for different parts of this problem.
 - Different items can be picked up from **racks** and examined. The user can request for the **catalog** associated with the items by clicking on the item.
 - Related items can be picked from the **drawers** of an item cabinet.
 - The items can be organized in the form of a book, similar to the way information about electronic components are organized in a semiconductor hand book.
- Once the users make up their mind about an item they wish to buy, they can put them into a **shopping cart**.

User interface inspection

- Nielson [Niel94] studied common usability problems and built a check list of points which can be easily checked for an interface. The following check list is based on the work of Nielson [Niel94].

Visibility of the system status. The system should as far as possible keep the user informed about the status of the system and what is going on.

Match between the system and the real world. The system should speak the user's language words, phrases, and concepts familiar to that used by the user, rather than using system-oriented terms.

Undoing mistakes. The user should feel that he is in control rather than feeling helpless or to be at the control of the system. An important step toward this is that the users should be able to undo and redo operations.

Consistency. The user should not have to wonder whether different words, concepts, and operations mean the same thing in different situations.

Recognition rather than recall. The user should not have to recall information which was presented in another screen. All data and instructions should be visible on the screen for selection by the user.

Support for multiple skill levels. Provision of accelerations for experienced users allows them to efficiently carry out the actions they frequently require to perform.

Aesthetic and minimalist design. Dialogs should not contain information which are irrelevant and are rarely needed. Every extra unit of information in a dialog competes with the relevant units and diminishes their visibility.

Help and error messages. These should be expressed in plain language (no codes), precisely indicating the problem, and constructively suggesting a solution.

Error prevention. Error possibilities should be minimized. A key principle in this regard is to prevent the user from entering wrong values. In situations where a choice has to be made from among a discrete set of values, the control should present only the valid values using a drop-down list, a set of option buttons or a similar

multichoice control. When a specific format is required for attribute data, the entered data should be validated when the user attempts to submit the data.

5.Explain Coding phase of software development in detail?

Coding

- The input to the coding phase is the design document.
- During coding phase:
 - modules identified in the design document are coded according to the module specifications.
- At the end of the design phase we have:
 - module structure (e.g. structure chart) of the system
 - module specifications:
 - data structures and algorithms for each module.
- Objective of coding phase:
 - transform design into code
 - unit test the code.

Coding Standards

- Good software development organizations require their programmers to:
 - adhere to some standard style of coding
 - called coding standards.
- Many software development organizations:
 - formulate their own coding standards that suits them most,
 - require their engineers to follow these standards rigorously.
- Advantage of adhering to a standard style of coding:
 - it gives a uniform appearance to the codes written by different engineers,
 - it enhances code understanding,
 - encourages good programming practices.
- A coding standard
 - sets out standard ways of doing several things:
 - the way variables are named,
 - code is laid out,
 - maximum number of source lines allowed per function, etc.

Coding guidelines

- Provide general suggestions regarding coding style to be followed:
 - leave actual implementation of the guidelines:
 - to the discretion of the individual engineers.

Code inspection and code walk throughs

- Detect as many errors as possible during inspection and walkthrough:
- detected errors require less effort for correction

- much higher effort needed if errors were to be detected during integration or system testing.
- After a module has been coded,
 - code inspection and code walk through are carried out
 - ensures that coding standards are followed
 - helps detect as many errors as possible before testing.

Coding Standards and Guidelines

- Good organizations usually develop their own coding standards and guidelines:
 - depending on what best suits their organization.
- We will discuss some representative coding standards and guidelines.

Representative Coding Standards

- Rules for limiting the use of globals:
 - what types of data can be declared global and what can not.
- Naming conventions for
 - global variables,
 - local variables, and
 - constant identifiers.
- Contents of headers for different modules:
 - The headers of different modules should be standard for an organization.
 - The exact format for header information is usually specified.
- Header data:
 - Name of the module,
 - date on which the module was created,
 - author's name,
 - modification history,
 - synopsis of the module,
 - different functions supported, along with their input/output parameters,
 - global variables accessed/modified by the module.
- Error return conventions and exception handling mechanisms.
 - the way error and exception conditions are handled should be standard within an organization.
 - For example, when different functions encounter error conditions
 - should either return a 0 or 1 consistently.

Representative Coding Guidelines

- Do not use too clever and difficult to understand coding style.
 - Code should be easy to understand.
- Many inexperienced engineers actually take pride:
 - in writing cryptic and incomprehensible code.
- Clever coding can obscure meaning of the code:
 - Hampers understanding.
 - Makes later maintenance difficult.
- Avoid obscure side effects.

- The side effects of a function call include:
 - modification of parameters passed by reference,
 - modification of global variables,
 - I/O operations.
- An obscure side effect:
 - one that is not obvious from a casual examination of the code.
- Obscure side effects make it difficult to understand a piece of code.
- For example,
 - if a global variable is changed obscurely in a called module,
 - it becomes difficult for anybody trying to understand the code.
- Do not use an identifier (variable name) for multiple purposes.
 - Programmers often use the same identifier for multiple purposes.
 - For example, some programmers use a temporary loop variable also for storing the final result.

Example use of a variable for multiple purposes

```
for(i=1;i<100;i++)
    {.....}
    i=2*p*q;
return(i);
```

- The rationale given by programmers for such use:
 - memory efficiency:
 - e.g. three variables use up three memory locations,
 - whereas the same variable used in three different ways uses just one memory location.
 - There are several things wrong with this approach:
 - hence should be avoided.
 - Each variable should be given a name indicating its purpose:
 - This is not possible if an identifier is used for multiple purposes.
 - Leads to confusion and annoyance
 - for anybody trying to understand the code.
 - Also makes future maintenance difficult.
- Code should be well-documented.
- Rules of thumb:
 - on the average there must be at least one comment line
 - for every three source lines.
 - The length of any function should not exceed 10 source lines.
- Lengthy functions:
 - usually very difficult to understand
 - probably do too many different things.

- Do not use goto statements.
- Use of goto statements:
 - make a program unstructured
 - make it very difficult to understand.

Code Walk Through

- An informal code analysis technique.
 - undertaken after the coding of a module is complete.
- A few members of the development team select some test cases:
 - simulate execution of the code by hand using these test cases.
- Even though an informal technique:
 - several guidelines have evolved over the years
 - making this naive but useful analysis technique more effective.
 - These guidelines are based on
 - personal experience, common sense, and several subjective factors.
- The guidelines should be considered as examples:
 - rather than accepted as rules to be applied dogmatically.
- The team performing code walk through should not be either too big or too small.
 - Ideally, it should consist of between three to seven members.
- Discussion should focus on discovery of errors:
 - and not on how to fix the discovered errors.
- To foster cooperation:
 - avoid the feeling among engineers that they are being evaluated in the code walk through meeting,
 - managers should not attend the walk through meetings.

Code Inspection

- In contrast to code walk throughs,
 - code inspection aims mainly at discovery of commonly made errors.
- During code inspection:

- the code is examined for the presence of certain kinds of errors,
- in contrast to the hand simulation of code execution done in code walk throughs.
- For instance, consider:
 - classical error of writing a procedure that modifies a formal parameter
 - while the calling routine calls the procedure with a constant actual parameter.
- It is more likely that such an error will be discovered:
 - by looking for this kind of mistakes in the code,
 - rather than by simply hand simulating execution of the procedure.
- Good software development companies:
 - collect statistics of errors committed by their engineers
 - identify the types of errors most frequently committed.
- A list of common errors:
 - can be used during code inspection to look out for possible errors.

Commonly made errors

- Use of uninitialized variables.
- Nonterminating loops.
- Array indices out of bounds.
- Incompatible assignments.
- Improper storage allocation and deallocation.
- Actual and formal parameter mismatch in procedure calls.
- Jumps into loops.

Code Inspection

- Use of incorrect logical operators
 - or incorrect precedence among operators.
- Improper modification of loop variables.
- Comparison of equality of floating point values, etc.
- Also during code inspection,
 - adherence to coding standards is checked.

5.Explain Software Documentation?

Software Documentation

- When developing a software product we develop various kinds of documents :
 - In addition to executable files and the source code:
 - users' manual,
 - software requirements specification (SRS) document,
 - design document, test document,
 - installation manual, etc.
- All these documents are a vital part of good software development practice.
- Good documents enhance understandability and maintainability of a software product.
- Different types of software documents can be classified into:
 - internal documentation,
 - external documentation (supporting documents).

Internal Documentation

- Internal documentation:
 - documentation provided in the source code itself.
- External documentation:
 - documentation other than those present in the source code.
- Internal documentation provided through:
 - use of meaningful variable names,
 - code indentation,
 - code structuring,
 - use of enumerated types and constant identifiers,
 - use of user-defined data types, etc.
 - module headers and comments

- Good software development organizations:
 - ensure good internal documentation
 - through coding standards and coding guidelines.
- Example of unhelpful documentation:
 - `a = 10; /* a made 10 */`
- Careful experimentation suggests:
 - meaningful variable names
is the most useful internal documentation.

External Documentation

- Users' manual,
- Software requirements specification document,
- Design document,
- Test documents,
- Installation instructions, etc.
- A systematic software development style ensures:
 - all external documents are produced in an orderly fashion.
- An important feature of good documentation is consistency.
- Unless all documents are consistent with each other,
- a lot of confusion is created for somebody trying to understand the product.
- All the documents for a product should be up-to-date:
- Even a few out-of-date documents can create severe confusion.

Textual Documents

- Readability is an important attribute of textual documents.
- Readability determines understandability
 - hence determines maintainability.
- A well-known readability measure of text documents:
 - Gunning's Fog Index.

Gunning's Fog Index

$$F=0.4 \frac{\text{Number of Words}}{\text{Number of Sentences}} + \frac{\text{Percentage of words of 3 or more syllables}}{100}$$

- F corresponds to the number of years of schooling to easily understand the document.
- A document is easy to understand if:
 - all sentences are small
 - use only 4 to 5 words each
 - small number of characters used per word:
 - normally not exceeding five or six characters.

6.Explain Unit Testing

Unit testing

- Unit testing is undertaken after a module has been coded and successfully reviewed. Unit testing (or module testing) is the testing of different units (or modules) of a system in isolation

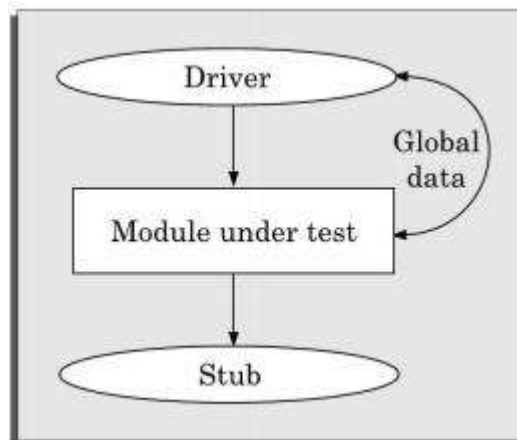


Fig: Unit testing with the help of driver and stub modules

- In order to test a single module, a complete environment is needed to provide all that is necessary for execution of the module.
- That is, besides the module under test itself, the following steps are needed in order to be able to test the module:
 - The procedures belonging to other modules that the module under test calls.
 - Nonlocal data structures that the module accesses.
 - A procedure to call the functions of the module under test with appropriate parameters.
- Modules required to provide the necessary environment (which either call or are called by the module under test) is usually not available until they too have been unit tested, stubs and drivers are designed to provide the complete environment for a module. The role of stub and driver modules is pictorially

shown in fig.

- A stub procedure is a dummy procedure that has the same I/O parameters as the given procedure but has a highly simplified behavior.
- For example, a stub procedure may produce the expected behavior using a simple table lookup mechanism.
- A driver module contains the nonlocal data structures accessed by the module under test, and would also have the code to call the different functions of the module with appropriate parameter values.

a) **Black box testing**

- In the black-box testing, test cases are designed from an examination of the input/output values only and no knowledge of design, or code is required.
- The following are the two main approaches to designing black box test cases.
 - Equivalence class partitioning
 - Boundary value analysis

Equivalence Class Partitioning

- In this approach, the domain of input values to a program is partitioned into a set of equivalence classes.
- This partitioning is done such that the behavior of the program is similar for every input data belonging to the same equivalence class. The main idea behind defining the equivalence classes is that testing the code with any one value belonging to an equivalence class is as good as testing the software with any other value belonging to that equivalence class.
 1. Equivalence classes for a software can be designed by examining the input data and output data. The following are some general guidelines for designing the equivalence classes: If the input data values to a system can be specified by a range of values, then one valid and two invalid equivalence classes should be defined.
 2. If the input data assumes values from a set of discrete members of some domain, then one equivalence class for valid input values and another equivalence class for invalid input values should be defined.

Example#1: For a software that computes the square root of an input integer which can assume values in the range of 0 to 5000, there are three equivalence classes: The set of negative integers, the set of integers in the range of 0 and 5000, and the integers larger than 5000. Therefore, the test cases must include representatives for each of the three equivalence classes and a possible test set can be: {-5, 500, 6000}.

Example#2: Design the black-box test suite for the following program. The program computes the intersection point of two straight lines and displays the result. It reads two integer pairs (m1, c1) and (m2, c2) defining the two straight lines of the form $y = mx + c$.

The equivalence classes are the following:

- Parallel lines ($m_1 = m_2, c_1 \neq c_2$)
- Intersecting lines ($m_1 \neq m_2$)
- Coincident lines ($m_1 = m_2, c_1 = c_2$)

Now, selecting one representative value from each equivalence class, the test suit (2, 2) (2, 5), (5, 5) (7, 7), (10, 10) (10, 10) are obtained.

Boundary Value Analysis

- A type of programming error frequently occurs at the boundaries of different equivalence classes of inputs.
- The reason behind such errors might purely be due to psychological factors. Programmers often fail to see the special processing required by the input values that lie at the boundary of the different equivalence classes.
- For example, programmers may improperly use < instead of <=, or conversely <= for <. Boundary value analysis leads to selection of test cases at the boundaries of the different equivalence classes.

Example: For a function that computes the square root of integer values in the range of 0 and 5000, the test cases must include the following values: {0, - 1,5000,5001}.

b) White-box testing(Structural testing or glass box testing or clear box testing) :

- White box testing is done to test internals of the program. This is done by examining the program structure and by deriving test cases from the program logic.
- White-box testing is an important type of unit testing
- A white-box testing strategy can either be coverage-based or fault-based

Fault-based testing

- A fault-based testing strategy targets to detect certain types of faults.
- An example of a fault-based strategy is mutation testing

Coverage-based testing

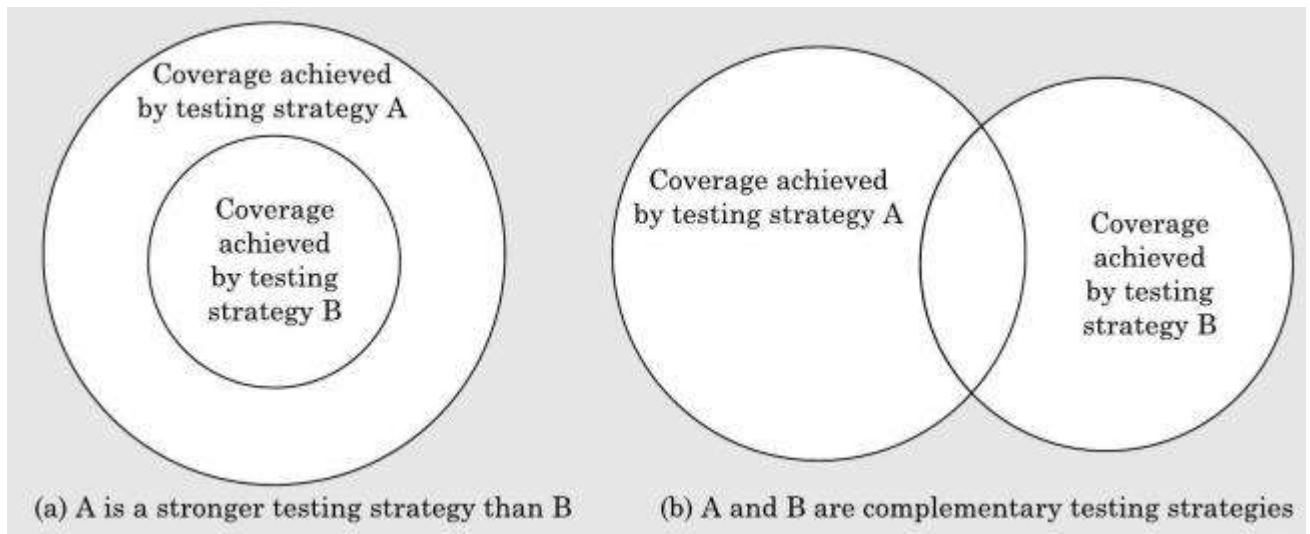
- A coverage-based testing strategy attempts to execute (or cover) certain elements of a program
- Examples of coverage-based testing strategies are statement coverage, branch coverage, and path coverage-based testing.

Testing criterion for coverage-based testing

- The set of specific program elements that a testing strategy requires to be executed is called the testing criterion of the strategy.

Stronger versus weaker testing

- A white-box testing strategy is said to be stronger than another strategy, if all types of program elements covered by the second testing strategy are also covered by the first testing strategy, and the first strategy additionally covers some more types of elements not covered by the second strategy.
- If a stronger testing has been performed, then a weaker testing need not be carried out.



a)Statement Coverage

- The statement coverage strategy aims to design test cases so that every statement in a program is executed at least once.
- The principal idea governing the statement coverage strategy is that unless a statement is executed, it is very hard to determine if an error exists in that statement

Example: Consider the Euclid's GCD computation algorithm:

```
int compute_gcd(x, y) int x, y;
{
1   while (x!= y){
2       if (x>y) then
3           x= x - y;
4       else y= y - x;
5   }
6   return x;
}
```

By choosing the test set $\{(x=3, y=3), (x=4, y=3), (x=3, y=4)\}$, we can exercise the program such that all statements are executed at least once.

b)Branch coverage

- In the branch coverage-based testing strategy, test cases are designed to make each branch condition to assume true and false values in turn.
- Branch testing is also known as **edge testing** as in this testing scheme, each edge of a program's control flow graph is traversed at least once.
- It is obvious that branch testing guarantees statement coverage and thus is **a stronger testing strategy compared to the statement coverage-based testing**.

- For Euclid's GCD computation algorithm, the test cases for branch coverage can be $\{(x=3, y=3), (x=3, y=2), (x=4, y=3), (x=3, y=4)\}$.

- **Theorem:**

Branch coverage-based testing is stronger than statement coverage-based testing.

Proof:

We need to show that

- a. Branch coverage ensures statement coverage, and
- b. Statement coverage does not ensure branch coverage

a) Branch testing would guarantee statement coverage since every statement belongs to some branch

b) To show that statement coverage does not ensure branch coverage, it would be sufficient to give an example of a test suite that achieves statement coverage, but does not cover at least one branch

c) Condition Coverage

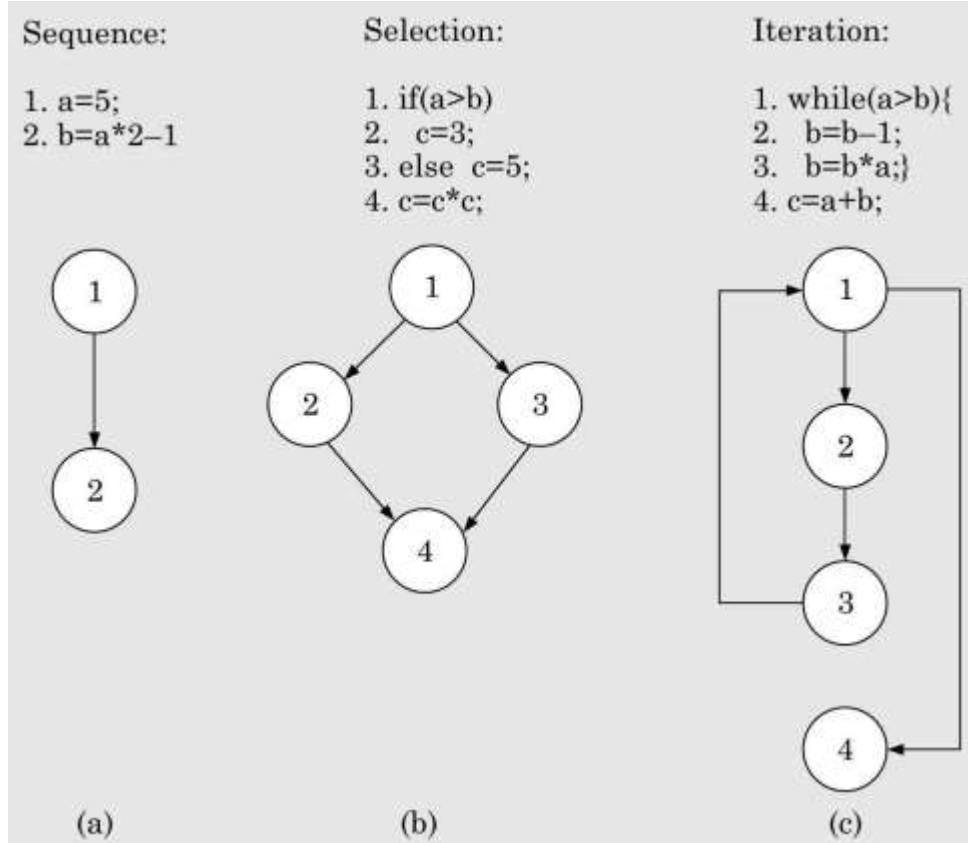
- Test cases are designed to make each component of a composite conditional expression to assume both true and false values.
- For example, in the conditional expression $((c1.and.c2).or.c3)$, the components $c1$, $c2$ and $c3$ are each made to assume both true and false values
- For a composite conditional expression of n components, for condition coverage, 2^n test cases are required

d) Path Coverage

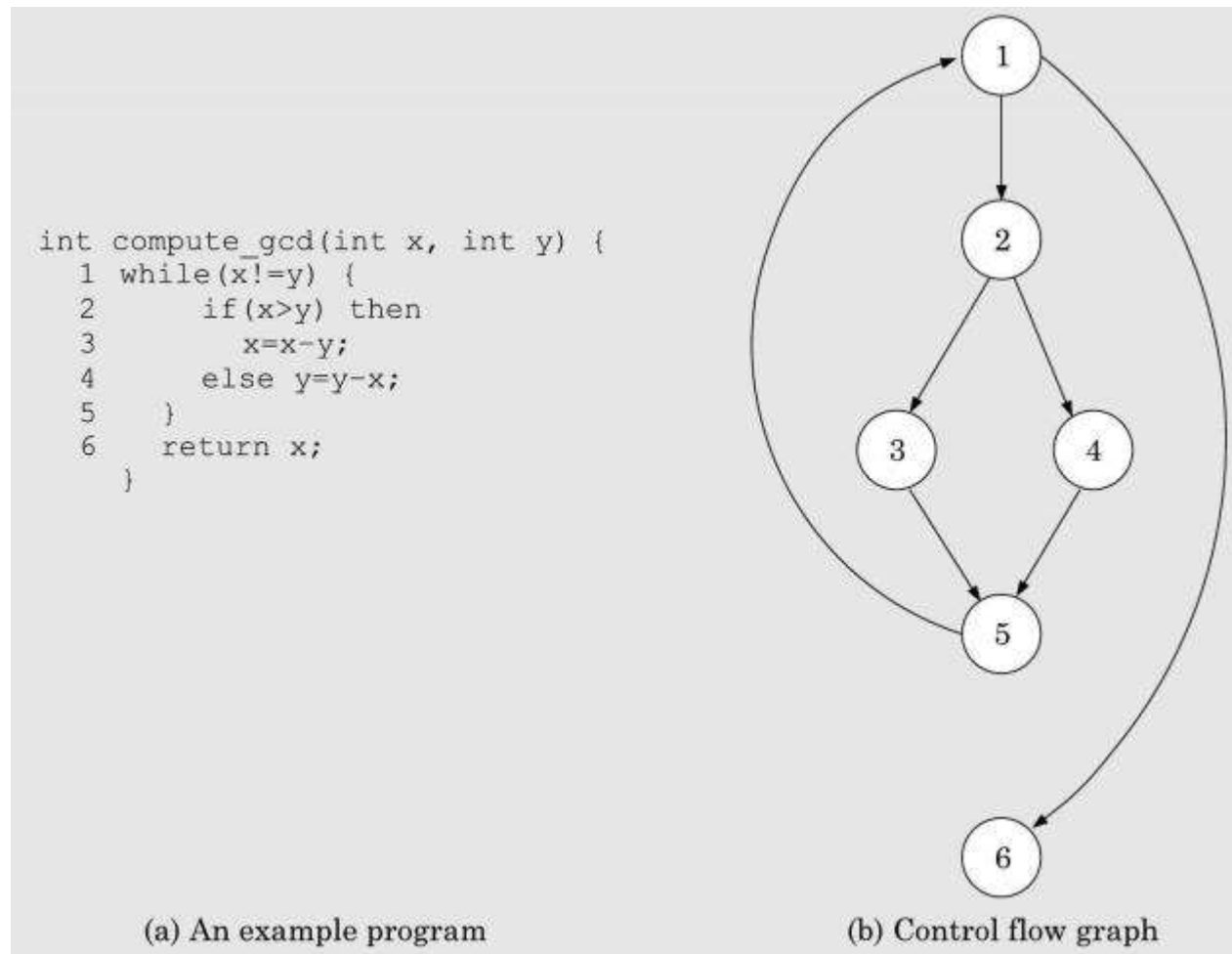
- The path coverage-based testing strategy requires us to design test cases such that all linearly independent paths in the program are executed at least once.
- A linearly independent path can be defined in terms of the control flow graph (CFG) of a program.

Control flow graph(CFG):

A control flow graph describes the sequence in which the different instructions of a program get executed.



CFG for a) sequence, b)selection and c)iteration type of constructs



Control flow diagram of an example program

Path:

- A path through a program is a node and edge sequence from the starting node to a terminal node of the control flow graph of a program.

Linearly independent path

- If a set of paths is linearly independent of each other, then no path in the set can be obtained through any linear operations (i.e. additions or subtractions) on the other paths in the set.

McCabe's Cyclomatic Complexity Metric

- It defines an upper bound for the number of linearly independent paths through a program.
- There are three different ways to compute the cyclomatic complexity

Method 1

- $V(G) = E - N + 2$
Where $V(G)$ is cyclomatic complexity, N is the number of nodes of the control flow graph and E is the number of edges in the control flow graph.
- For the CFG of example shown in fig. , $E=7$ and $N=6$. Therefore, the cyclomatic complexity = $7-6+2 = 3$

Method 2

- $V(G) = \text{Total number of bounded areas} + 1$

Method 3

- The cyclomatic complexity of a program can also be easily computed by computing the number of decision statements of the program.
- If N is the number of decision statement of a program, then the McCabe's metric is equal to $N+1$

The following is the sequence of steps that need **to be undertaken for deriving the path coverage-based test cases for a program**

- Draw control flow graph
- Determine $V(G)$. This gives the minimum number of test cases required to achieve path coverage
- Repeat
Test using a randomly designed set of test cases
Perform dynamic analysis to check the path coverage achieved.
Until at least 90 per cent path coverage is achieved.

Uses of cyclomatic complexity metric

1. Estimation of structural complexity of code. Cyclomatic complexity of a program is a measure of the psychological complexity or the level of difficulty in understanding the program
2. Estimation of testing effort – Cyclomatic complexity is a measure of the maximum number of basis paths
3. Estimation of program reliability

e) Data Flow-based Testing

- Data flow-based testing method selects test paths of a program according to the definitions and uses of different variables in a program
- Consider a program P. For a statement numbered S, let
- **$DEF(S) = \{X/\text{statement S contains a definition of X}\}$, and**
 $USES(S) = \{X/\text{statement S contains a use of X}\}$
- For the statement **$S: a=b+c;$** , $DEF(S) = \{a\}$. $USES(S) = \{b, c\}$. The definition of variable X at statement S is said to be live at statement S1, if there exists a path from statement S to statement S1 which does not contain any definition of X.

f) Mutation testing

- In mutation testing, the software is first tested by using an initial test suite built up from the different white box testing strategies. After the initial testing is complete, mutation testing is taken up.
- The idea behind mutation testing is **to make few arbitrary changes to a program at a time.**
- Each time the program is changed, it is called as a **mutated program** and the change effected is called as **a mutant.**
- A mutated program is tested against the full test suite of the program. If there exists at least one test case in the test suite for which a mutant gives an incorrect result, then the mutant is said to be dead. If a mutant remains alive even after all the test cases have been exhausted, the test data is enhanced to kill the mutant.

- The process of generation and killing of mutants can be automated by predefining a set of primitive changes that can be applied to the program. These primitive changes can be alterations such as changing an arithmetic operator, changing the value of a constant, changing a data type, etc. A major disadvantage of the mutation-based testing approach is that it is computationally very expensive, since a large number of possible mutants can be generated.

7. Explain Debugging and program analysis tools.

Debugging

- Identifying errors in a program code and then fix them up are known as debugging.

Need for debugging

- Once errors are identified in a program code, it is necessary to first identify the precise program statements responsible for the errors and then to fix them.

Debugging approaches

The following are some of the approaches popularly adopted by programmers for debugging.

Brute Force Method:

- This is the most common method of debugging but is the least efficient method.
- In this approach, the program is loaded with print statements to print the intermediate values with the hope that some of the printed values will help to identify the statement in error.
- This approach becomes more systematic with the use of a symbolic debugger (also called a source code debugger), because values of different variables can be easily checked and break points and watch points can be easily set to test the values of variables effortlessly.

Backtracking:

- This is also a fairly common approach. In this approach, beginning from the statement at which an error symptom has been observed, the source code is traced backwards until the error is discovered.
- Unfortunately, as the number of source lines to be traced back increases, the number of potential backward paths increases and may become unmanageably large thus limiting the use of this approach.

Cause Elimination Method:

- In this approach, a list of causes which could possibly have contributed to the error symptom is developed and tests are conducted to eliminate each.
- A related technique of identification of the error from the error symptom is the software fault tree analysis.

Program Slicing:

- This technique is similar to back tracking.
- The search space is reduced by defining slices.
- A slice of a program for a particular variable at a particular statement is the set of source lines preceding this statement that can influence the value of that variable

Debugging guidelines

- Debugging is often carried out by programmers based on their ingenuity. The following are some

general guidelines for effective debugging:

- Many times debugging requires a thorough understanding of the program design. Trying to debug based on a partial understanding of the system design and implementation may require an inordinate amount of effort to be put into debugging even simple problems.
- Debugging may sometimes even require full redesign of the system. In such cases, a common mistakes that novice programmers often make is attempting not to fix the error but its symptoms.
- One must be beware of the possibility that an error correction may introduce new errors. Therefore after every round of error-fixing, regression testing must be carried out.

PROGRAM ANALYSIS TOOLS

- A program analysis tool means an automated tool that takes the source code or the executable code of a program as input and produces reports regarding several important characteristics of the program, such as its size, complexity, adequacy of commenting, adherence to programming standards, etc.
- Two broad categories of program analysis tools
 - Static Analysis tools
 - Dynamic Analysis tools

Static program analysis tools

- Static analysis tool is also a program analysis tool. It assesses and computes various characteristics of a software product without executing it.
- Typically, static analysis tools analyze some structural representation of a program to arrive at certain analytical conclusions, e.g. that some structural properties hold. The structural properties that are usually analyzed are:
 - Whether the coding standards have been adhered to?
 - Certain programming errors such as uninitialized variables and mismatch between actual and formal parameters, variables that are declared but never used are also checked.
- Code walk throughs and code inspections might be considered as static analysis methods.
- Used to denote automated analysis tools - a compiler can be considered to be a static program analysis tool.

Dynamic program analysis tools

- Dynamic program analysis techniques require the program to be executed and its actual behavior recorded.
- A dynamic analyzer usually instruments the code
- The instrumented code when executed allows us to record the behavior of the software for different test cases.
- After the software has been tested with its full test suite and its behavior recorded, the dynamic analysis tool carries out a post execution analysis and produces reports which describe the structural coverage that has been achieved by the complete test suite for the program.
- For example, the post execution dynamic analysis report might provide data on extent statement, branch and path coverage achieved.
- The dynamic analysis results are reported in the form of a histogram or a pie chart to describe the structural coverage achieved for different modules of the program.
- The output of a dynamic analysis tool can be stored and printed easily and provides evidence that

thorough testing has been done

- The dynamic analysis results the extent of testing performed in white-box mode. If the testing coverage is not satisfactory more test cases can be designed and added to the test suite.
- Dynamic analysis results can help to eliminate redundant test cases from the test suite.

8. Integration and system Testing

Integration testing

- The primary objective of integration testing is to test the module interfaces, i.e. there are no errors in the parameter passing, when one module invokes another module.
- During integration testing, different modules of a system are integrated in a planned manner using an integration plan.
- The integration plan specifies the steps and the order in which modules are combined to realize the full system.
- After each integration step, the partially integrated system is tested. An important factor that guides the integration plan is the module dependency graph.
- The structure chart (or module dependency graph) denotes the order in which different modules call each other. By examining the structure chart the integration plan can be developed.

Integration test approaches

- There are four types of integration testing approaches
 - Big bang approach
 - Top-down approach
 - Bottom-up approach
 - Mixed-approach
- **Big-Bang Integration Testing**
 - It is the simplest integration testing approach, where all the modules making up a system are integrated in a single step.
 - Only for very small systems.
 - Debugging errors reported during big bang integration testing are very expensive to fix.
- **Bottom-Up Integration Testing**
 - In bottom-up testing, each subsystem is tested separately and then the full system is tested.
 - A subsystem might consist of many modules which communicate among each other through well-defined interfaces.
 - The primary purpose of testing each subsystem is to test the interfaces among various modules making up the subsystem.
 - Both control and data interfaces are tested.
 - Advantage of bottom-up integration testing is that several disjoint subsystems can be tested simultaneously
 - Disadvantage of bottom-up testing is the complexity that occurs when the system is made up of a large number of small subsystems.
- **Top-Down Integration Testing**
 - Top-down integration testing starts with the main routine and one or two subordinate routines in the system.
 - After the top-level 'skeleton' has been tested, the immediately subroutines of the skeleton' are

combined with it and tested.

- A pure top-down integration does not require any driver routines.
- A disadvantage of the top-down integration testing approach is that in the absence of lower-level routines, many times it may become difficult to exercise the top-level routines in the desired manner since the lower-level routines perform several low-level functions such as I/O.

- **Mixed Integration Testing**

- A mixed (sandwiched) integration testing follows a combination of top-down and bottom-up testing approaches.
- In top-down approach, testing can start only after the top-level modules have been coded and unit tested. Similarly, bottom-up testing can start only after the bottom level modules are ready.
- The mixed approach overcomes this shortcoming of the top-down and bottom-up approaches.
- In the mixed testing approaches, testing can start as and when modules become available.
- Therefore, this is one of the most commonly used integration testing approaches.

Phased vs. incremental testing

- The different integration testing strategies are either phased or incremental.
- A comparison of these two strategies is as follows:
 - In incremental integration testing, only one new module is added to the partial system each time.
 - In phased integration, a group of related modules are added to the partial system each time.
- Phased integration requires less number of integration steps compared to the incremental integration approach.

However, when failures are detected, it is easier to debug the system in the incremental testing approach since it is known that the error is caused by addition of a single module

SYSTEM TESTING

- System tests are designed to validate a fully developed system to assure that it meets its requirements.
- There are essentially three main kinds of system testing:
 - Alpha Testing refers to the system testing carried out by the test team within the developing organization.
 - Beta testing is the system testing performed by a select group of friendly customers.
 - Acceptance testing is the system testing performed by the customer to determine whether he should accept the delivery of the system.
- The system test cases can be classified into functionality and performance test cases.
- **Smoke testing** is done to check whether at least the main functionalities of the software are working properly. It is performed before system testing.
- The functionality tests test the functionality of the software to check whether it satisfies the functional requirements as documented in the SRS document.
- The performance tests test the conformance of the system with the nonfunctional requirements of the system.

Performance testing

- Performance testing is carried out to check whether the system needs the non-functional requirements

identified in the SRS document.

- There are several types of performance testing.
 - Stress testing
 - Volume testing
 - Configuration testing
 - Compatibility testing
 - Regression testing
 - Recovery testing
 - Maintenance testing
 - Documentation testing
 - Usability testing

Stress Testing(endurance testing):

- Stress testing evaluates system performance when it is stressed for short periods of time.
- Stress tests are black box tests which are designed to impose a range of abnormal and even illegal input conditions so as to stress the capabilities of the software.
- Input data volume, input data rate, processing time, utilization of memory, etc. are tested beyond the designed capacity.
- For example, if the non-functional requirement specification states that the response time should not be more than 20 secs per transaction when 60 concurrent users are working, then during the stress testing the response time is checked with 60 users working simultaneously.

Volume Testing

- To check whether the data structures have been designed to successfully extraordinary situations.
- For example, a compiler might be tested to check whether the symbol table overflows when a very large program is compiled.

Configuration Testing

- To analyze system behavior in various hardware and software configurations specified in the requirements.
- Sometimes systems are built in variable configurations for different users.
- For example, a minimal system to serve a single user, and other extension configurations to serve additional users.

Compatibility Testing

- It is required when the system interfaces with other types of systems.
- Aims to check whether the interface functions perform as required.
- For instance, if the system needs to communicate with a large database system to retrieve information, compatibility testing is required to test the speed and accuracy of data retrieval.

Regression Testing

- It is required when the system being tested is an upgradation of an already existing system to fix some bugs or enhance functionality, performance, etc.
- Regression testing is the practice of running an old test suite after each change to the system or after each bug fix to ensure that no new bug has been introduced due to the change or the bug fix

Recovery Testing

- Recovery testing tests the response of the system to the presence of faults, or loss of power, devices, services, data, etc.

- The system is subjected to the loss of the mentioned resources (as applicable and discussed in the SRS document) and it is checked if the system recovers satisfactorily.
- For example, the printer can be disconnected to check if the system hangs. Or, the power may be shut down to check the extent of data loss and corruption.

Maintenance Testing

- Addresses the diagnostic programs, and other procedures that are required to be developed to help maintenance of the system.
- It is verified that the artifacts exist and they perform properly.

Documentation Testing

- It is checked that the required user manual, maintenance manuals, and technical manuals exist and are consistent.
- If the requirements specify the types of audience for which a specific manual should be designed, then the manual is checked for compliance.

Usability Testing

- Usability testing concerns checking the user interface to see if it meets all user requirements concerning the user interface.
- During usability testing, the display screens, report formats, and other aspects relating to the user interface requirements are tested.

Error seeding

- Sometimes the customer might specify the maximum number of allowable errors that may be present in the delivered system.
- These are often expressed in terms of maximum number of allowable errors per line of source code.
- Error seed can be used to estimate the number of residual errors in a system.

General Issues associated with Testing

- How to document the results of testing and how to perform regression testing
- Test summary Report:
 - A piece of documentation produced towards the end of testing .
 - It specifies what is the total number of tests that were applied to a subsystem, Out of the total number of tests how many tests were successful, and How many were unsuccessful, and the degree to which they were unsuccessful.
- Regressing testing is the practice of running an old test suite after each change to the system or after each bug fix to ensure that no new bug has been introduced due to the change or the bug fix.

9. Explain testing object-oriented programs?

TESTING OBJECT-ORIENTED PROGRAMS

- Testing object-oriented programs is much more difficult and required much more cost and effort as compared to testing similar procedural programs.
- Additional test cases are needed to be designed to detect bugs.
- Basic issues in testing object-oriented programs
 - What is a Suitable Unit for Testing Object-oriented Programs?
 - Do various Object-orientation features make testing easy?
 - Why are traditional techniques considered not satisfactory for testing object-oriented programs?
 - Grey-Box testing of objects oriented programs?
- What is a Suitable Unit for Testing Object-oriented Programs?
 - Do various Object-orientation features make testing easy?
 - Why are traditional techniques considered not satisfactory for testing object-oriented programs?
 - Grey-Box testing of objects oriented programs?

Grey-Box Testing of Object-oriented programs

- For object-oriented programs, several types of test cases can be designed based on the design models of object-oriented programs. These are called the grey-box test cases.
- Types of grey-box testing based on UML models
 - State-model-based testing
 - Use case-based testing
 - Class diagram-based testing
 - Sequence diagram-based testing
- State-model based testing
 - State coverage : Each method of an object are tested at each state of the object
 - State transition coverage: It is tested whether all transitions depicted in the state model work satisfactorily
 - State transition path coverage: All transition paths in the state model are tested.
- Use case based testing
 - Scenario coverage: Each use case consists of a mainline scenario and several alternate scenarios.
 - For each use case, the mainline and all alternate sequences are tested to check if any errors show up
- Class diagram-based testing
 - Testing derived classes: All derived classes of the base class have to be instantiated and tested.
 - Association testing: All association relations are tested
 - Aggregation testing: Various aggregate objects are created and tested.
- Sequence diagram-based testing
 - Method coverage: All methods depicted in the sequence diagrams are covered.

- Message path coverage: All message paths that can be constructed from the sequence diagrams are covered.

Integration Testing of Object-oriented Programs

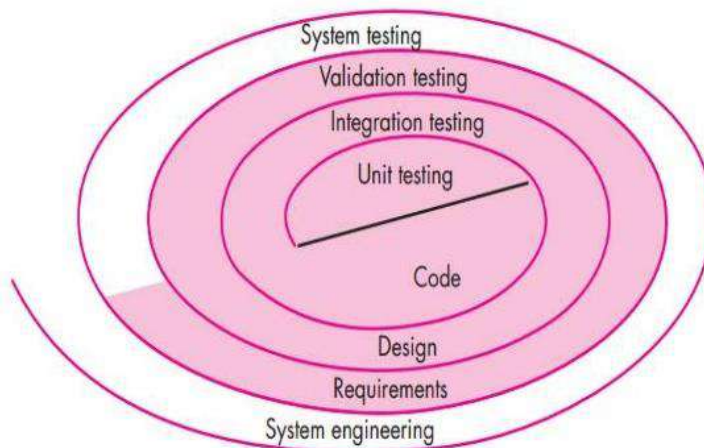
- Two main approaches
 - Thread-based
 - Use based
- Thread-based approach
 - In this approach, all classes that need to collaborate to realize the behavior of a single use case are integrated and tested.
 - After all the required classes for a use case are integrated and tested, another use case is taken up and other classes necessary for execution of the second use case to run are integrated and tested.
 - This is continued till all use cases have been considered.
- Use-based approach:
 - Used-based integration begins by testing classes that either need no service from other classes or need services from at most a few other classes.
 - After these classes have been integrated and tested, classes that use the services from the already integrated classes are integrated and tested
 - This is continued till all the classes have been integrated and tested..

10. Discuss in detail about test strategies for conventional software (Nov 2017)

- A Strategic Approach to Testing
 - To perform effective testing, a software team should conduct effective formal technical reviews
 - Testing begins at the component level and work outward toward the integration of the entire computer based system
 - Different testing techniques are appropriate at different points in time
 - Testing is conducted by the developer of the software and (for large projects) by an independent test group
 - Testing and debugging are different activities, but debugging must be accommodated in any testing strategy
- Verification and Validation
 - Software testing is part of a broader group of activities called verification and validation that are involved in software quality assurance
 - Verification (Are the algorithms coded correctly?) – The set of activities that ensure that software correctly implements a specific function or algorithm
 - Validation (Does it meet user requirements?) – The set of activities that ensure that the software that has been built is traceable to customer requirements.
 - Verification: “Are we building the product right?”
 - Validation: “Are we building the right product?”
- Organizing for Software Testing

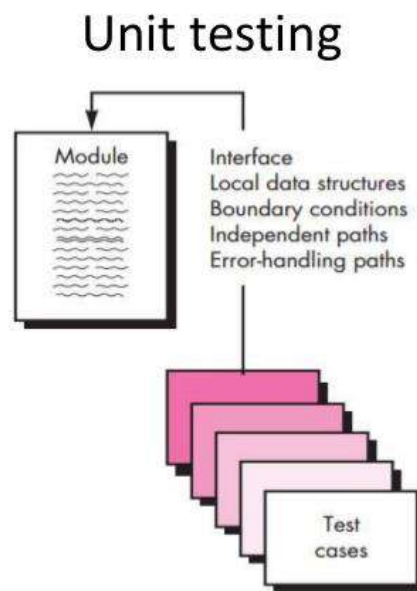
- Testing should aim at "breaking" the software
 - Common misconceptions – The developer of software should do no testing at all
 - that the software should be “tossed over the wall” to strangers who will test it mercilessly,
 - that testers get involved with the project only when the testing steps are about to begin.
 - software architecture is complete does an independent test group become involved.
 - The role of an independent test group (ITG) is to remove the inherent problems associated with letting the builder test the thing that has been built. Independent testing removes the conflict of interest that may otherwise be present.
- Software testing strategy
 - Unit testing – Concentrates on each component/function of the software as implemented in the source code
 - Integration testing – Focuses on the design and construction of the software architecture
 - Validation testing – Requirements are validated against the constructed software
 - System testing – The software and other system elements are tested as a whole

Testing strategy



- Criteria for Completion of testing when is testing completed ?
 - A classic question arises every time software testing is discussed: “When are we done testing—how do we know that we’ve tested enough?” Sadly, there is no definitive,
 - answer to this question, but there are a few pragmatic responses and early attempts at empirical guidance.
 - By collecting metrics during software testing and making use of existing software reliability models, it is possible to develop meaningful guidelines for answering the question: “When are we done testing?”
- STRATEGIC ISSUES
 - Specify product requirements in a quantifiable manner long before testing commences.

- State testing objectives explicitly.
- Understand the users of the software and develop a profile for each user category.
- Develop a testing plan that emphasizes “rapid cycle testing.”
- Test Strategies for Conventional Software Unit testing
 - Focuses testing on the function or software module
 - Concentrates on the internal processing logic and data structures
 - Is simplified when a module is designed with high cohesion
 - Reduces the number of test cases
 - Allows errors to be more easily predicted and uncovered
 - Concentrates on critical modules and those with high cyclomatic complexity when testing resources are limited

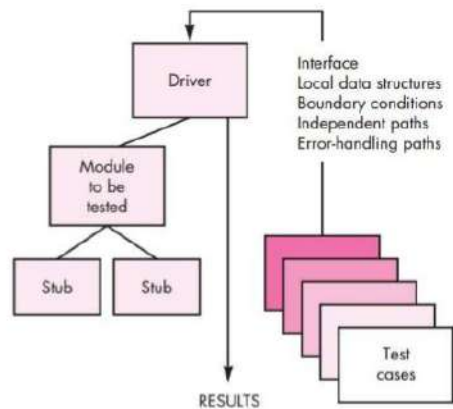


11

- Unit testing considerations
 - Module interface – Ensure that information flows properly into and out of the module
 - Local data structures – Ensure that data stored temporarily maintains its integrity during all steps in an algorithm execution
 - Boundary conditions – Ensure that the module operates properly at boundary values established to limit or restrict processing
 - Independent paths (basis paths) – Paths are exercised to ensure that all statements in a module have been executed at least once
 - Error handling paths – Ensure that the algorithms respond correctly to specific error conditions
- Unit test procedures
 - Driver – A simple main program that accepts test case data, passes such data to the component being tested, and prints the returned results

- Stubs – Serve to replace modules that are subordinate to (called by) the component to be tested – It uses the module’s exact interface, may do minimal data manipulation, provides verification of entry, and returns control to the module undergoing testing
- Drivers and stubs both represent testing overhead. – Both must be written but don’t constitute part of the installed software product

Unit-test environment

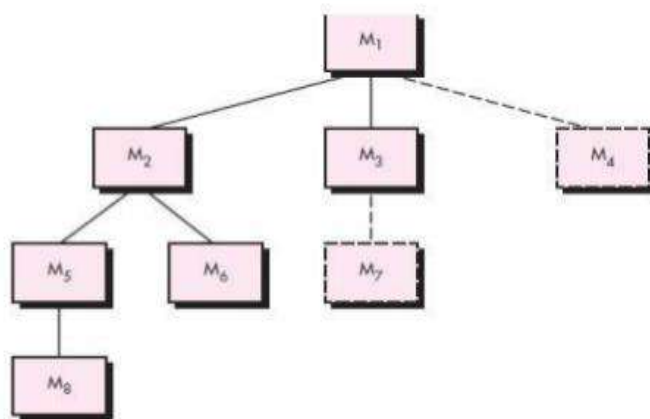


34

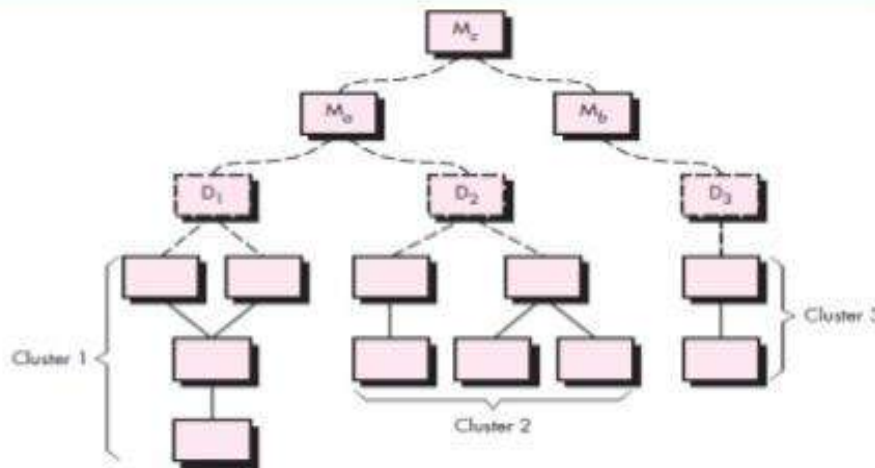
- Integration testing
 - Defined as a systematic technique for constructing the software architecture – At the same time integration is occurring, conduct tests to uncover errors associated with interfaces
 - Objective is to take unit tested modules and build a program structure based on the prescribed design
 - Two Approaches – Non-incremental Integration Testing – Incremental Integration Testing
- Non-incremental Integration Testing
 - Uses “Big Bang” approach • All components are combined in advance
 - The entire program is tested as a whole Chaos results
 - Many seemingly-unrelated errors are encountered
 - Correction is difficult because isolation of causes is complicated
 - Once a set of errors are corrected, more errors occur, and testing appears to enter an endless loop
- Incremental Integration Testing
 - The program is constructed and tested in small increments
 - Errors are easier to isolate and correct
 - Interfaces are more likely to be tested completely
 - A systematic test approach is applied
 - Different incremental integration strategies
 - Top-down integration
 - Bottom-up integration
 - Regression testing
 - Smoke testing

- Top-down Integration

- Modules are integrated by moving downward through the control hierarchy, beginning with the main module
- Subordinate modules are incorporated in two ways : – depth-first : All modules on a major control path are integrated – breadth-first : All modules directly subordinate at each level are integrated
- Advantages – This approach verifies major control or decision points early in the test process
- Disadvantages – Stubs need to be created to substitute for modules that have not been built or tested yet; this code is later discarded – Because stubs are used to replace lower level modules, no significant data flow can occur until much later in the integration/testing process



- For example, selecting the left-hand path, components M1, M2 , M5 would be integrated first. Next, M8 or (if necessary for proper functioning of M2) M6 would be integrated. Then, the central and right-hand control paths are built.
- Bottom-up Integration
 - Integration and testing starts with the most atomic modules in the control hierarchy
 - Advantages
 - This approach verifies low-level data processing early in the testing process
 - Need for stubs is eliminated
 - Disadvantages
 - Driver modules need to be built to test the lower-level modules; this code is later discarded or expanded into a full-featured version
 - Drivers inherently do not contain the complete algorithms that will eventually use the services of the lower-level modules; consequently, testing may be incomplete or more testing may be needed later when the upper level modules are available



- Integration follows the pattern illustrated in Figure Components are combined to form clusters 1, 2, and 3. Each of the clusters is tested using a driver (shown as a dashed block). Components in clusters 1 and 2 are subordinate to Ma. Drivers D1 and D2 are removed and the clusters are interfaced directly to Ma. Similarly, driver D3 for cluster 3 is removed prior to integration with module Mb. Both Ma and Mb will ultimately be integrated with component Mc, and so forth.
- Regression Testing
 - Each new addition or modification of data may cause problems with functions that previously worked flawlessly
 - Regression testing re-executes a small subset of tests that have already been conducted
 - Ensures that changes have not propagated unintended side effects
 - Helps to ensure that changes do not introduce unintended behavior or additional errors
 - May be done manually or through the use of automated capture/playback tools
 - Regression test suite contains three different classes of test cases
 - A representative sample of tests that will exercise all software functions
 - Additional tests that focus on software functions that are likely to be affected by the change
 - Tests that focus on the actual software components that have been changed
- Smoke testing
 - Designed as a pacing mechanism for time-critical projects – Allows the software team to assess its project on a frequent basis
 - Includes the following activities
 - The software components that have been translated into code and linked into a build
 - A series of breadth tests is designed to expose errors that will keep the build from properly performing its function
 - The goal is to uncover “show stopper” errors that have the highest likelihood of throwing the software project behind schedule – The build is integrated with other builds and the entire product is smoke tested daily

- Benefits of Smoke Testing
 - Integration risk is minimized – Daily testing uncovers incompatibilities and show- stoppers early in the testing process, thereby reducing schedule impact
 - The quality of the end-product is improved – Smoke testing is likely to uncover both functional errors and architectural and component-level design errors
 - Error diagnosis and correction are simplified – Smoke testing will probably uncover errors in the newest components that were integrated
 - Progress is easier to assess
 - As integration testing progresses, more software has been integrated and more has been demonstrated to work
 - Managers get a good indication that progress is being made
- Comparison b/w conventional & oo s/w Unit Testing
 - In conventional software we are testing the individual units/modules...
 - In object oriented software testing the class individual classes & subclasses..
 - which tends to focus on the algorithmic detail of a module and the data that flow across the module interface.
 - OO software is driven by the operations encapsulated by the class and the state behavior of the class